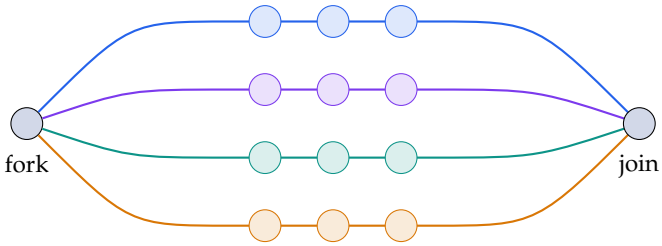
CS4533
Parallel and Concurrent Programming

Study Notes - Part 2



University of Moratuwa
Semester 7

Lectures 10–14

Distributed-memory programming with MPI · Deadlocks
Concurrent data structures · Solution patterns · Embarrassingly parallel problems

Message passing, deadlock, concurrent data structures and the patterns that organise parallel programs.

Contents

10 Distributed Memory Programming with MPI **1**

10.1 Distributed Memory Systems . . . 1

10.2 Why MPI? . . . 2

10.3 Our First MPI Program . . . 2

10.3.1 Compilation . . . 3

10.3.2 Execution . . . 3

10.3.3 MPI Programs — Conventions . . . 4

10.4 The Six Golden MPI Functions . . . 4

10.4.1 `MPI_Init` and `MPI_Finalize` . . . 4

10.5 Communicators . . . 5

10.6 Single-Program Multiple-Data (SPMD) . . . 5

10.7 Point-to-Point Communication . . . 5

10.7.1 `MPI_Send` . . . 5

10.7.2 Data Types . . . 6

10.7.3 `MPI_Recv` . . . 6

10.7.4 Message Matching . . . 7

10.7.5 Receiving Messages Without Knowing Everything . . . 7

10.7.6 Issues with Send and Receive . . . 8

10.8 Case Study: The Trapezoidal Rule in MPI . . . 8

10.8.1 Serial Pseudo-code . . . 9

10.8.2 Parallelising: Tasks and Communication . . . 9

10.8.3 Parallel Pseudo-code . . . 9

10.8.4 First Version . . . 10

10.9 Collective Communication . . . 11

10.9.1 Tree-Structured Global Sum . . . 11

10.9.2 `MPI_Reduce` . . . 12

10.9.3 Collective vs Point-to-Point Communication . . . 12

10.9.4 `MPI_Allreduce` . . . 13

10.9.5 Broadcast . . . 14

10.10 Data Distributions . . . 15

10.10.1 Example: Computing a Vector Sum . . . 15

10.10.2 Partitioning Options . . . 15

10.10.3 Parallel Implementation . . . 16
10.10.4 `MPI_Scatter` . . . 16
10.10.5 `MPI_Gather` . . . 17
10.10.6 `MPI_Allgather` . . . 17
10.11 Summary of Collectives . . . 18
10.12 Summary . . . 18

11 Deadlocks **20**
11.1 Resources . . . 20
11.1.1 Two Types of Resources . . . 21
11.2 Deadlocks . . . 21
11.2.1 Four Conditions for Deadlock . . . 21
11.3 Deadlock Modelling . . . 22
11.3.1 Example: How Deadlock Occurs . . . 22
11.4 Strategies for Dealing with Deadlocks . . . 23
11.5 Strategy 1: The Ostrich Algorithm . . . 23
11.6 Strategy 2: Detection and Recovery . . . 23
11.6.1 Detection with One Resource of Each Type . . . 24
11.6.2 Detection with Multiple Resources of Each Type . . . 24
11.6.3 Recovery from Deadlock . . . 26
11.7 Strategy 3: Deadlock Avoidance . . . 26
11.7.1 How Deadlock Can Be Avoided . . . 26
11.7.2 Resource Trajectories . . . 27
11.7.3 Safe and Unsafe States . . . 27
11.7.4 The Banker's Algorithm (Dijkstra) . . . 28
11.8 Strategy 4: Deadlock Prevention . . . 29
11.8.1 Attacking the Mutual Exclusion Condition . . . 29
11.8.2 Attacking the Hold and Wait Condition . . . 30
11.8.3 Attacking the No Preemption Condition . . . 30
11.8.4 Attacking the Circular Wait Condition . . . 30
11.8.5 Summary of Prevention Approaches . . . 31
11.9 Summary . . . 31

12 Concurrent Data Structures **33**
12.1 Motivation . . . 33
12.1.1 Example: `getAndIncrement()` . . . 33
12.1.2 Performance Problems with Locking . . . 34
12.2 Blocking Techniques . . . 34

12.2.1 The Combining Tree . . . 35
12.3 Non-blocking Techniques . . . 36
12.3.1 Non-blocking Progress Conditions . . . 36
12.3.2 Compare-and-Swap (CaS) . . . 37
12.3.3 Shared Counter Based on CaS . . . 37
12.4 Blocking vs Non-blocking Methods . . . 38
12.5 A Bounded Queue (Two Locks) . . . 38
12.5.1 The Enqueuer . . . 39
12.5.2 The Dequeuer . . . 40
12.5.3 The Code Skeleton . . . 40
12.6 Monitor Locks and Conditions . . . 41
12.6.1 The Lost Wake-Up Problem . . . 42
12.7 Split Counter . . . 42
12.8 A Lock-Free Queue . . . 43
12.8.1 Lock-Free Dequeue . . . 44
12.9 Summary . . . 44

13 Solution Patterns for Parallel Programming 46
13.1 Building a Solution by Composition . . . 46
13.2 Designing Parallel Algorithms . . . 46
13.2.1 Methodical Design (Foster's PCAM) . . . 47
13.2.2 Parallel Algorithm Design Issues . . . 48
13.2.3 Efficiency = Maximising Parallelism? . . . 48
13.3 Solution Patterns for Parallelism . . . 48
13.3.1 Loop Parallel . . . 49
13.3.2 Fork/Join . . . 50
13.3.3 Divide and Conquer . . . 50
13.3.4 Producer/Consumer . . . 51
13.3.5 Pipeline . . . 51
13.3.6 Asynchronous Agents . . . 52
13.4 Three Ways to Parallelise . . . 53
13.5 Load Balancing . . . 53
13.5.1 Task Queues . . . 54
13.5.2 Work Stealing . . . 54
13.6 Summary . . . 55

14 Embarrassingly Parallel Problems 57
14.1 Embarrassingly Parallel Problems . . . 57

14.2 MapReduce . . . 57
14.2.1 Higher-Order Functions . . . 58
14.2.2 The Two Functions . . . 58
14.2.3 Example: Word Counting . . . 59
14.2.4 Why MapReduce? . . . 59
14.2.5 MapReduce Implementations . . . 59
14.3 MapReduce with Hadoop . . . 60
14.3.1 The Data Model . . . 60
14.3.2 Execution on a Cluster/Cloud . . . 61
14.3.3 MapReduce Execution (Dean et al.) . . . 61
14.4 Designing MapReduce Applications . . . 62
14.5 Examples . . . 62
14.5.1 Word Count . . . 62
14.5.2 Mean . . . 62
14.5.3 Inverted Index . . . 63
14.5.4 Distributed Grep . . . 63
14.6 Summary . . . 64

A Quick Reference **65**
A.1 MPI Cheat Sheet . . . 65
A.2 Deadlock at a Glance . . . 65
A.3 Progress Conditions . . . 66
A.4 Pattern Selector . . . 66
A.5 Building These Notes . . . 66

List of Figures

10.1 A distributed-memory system: each CPU can directly access only its own memory; everything else travels over the interconnect. 1

10.2 Hello world: ranks $1..p-1$ send a greeting; rank 0 receives them in rank order. . . 3

10.3 A communicator numbers its processes with ranks $0 \ldots p-1$. 5

10.4 A message sent by q is received by r only if destination, source, tag and communicator all agree. 7

10.5 Whether `MPI_Send` returns early depends on the message size and the implementation. 8

10.6 The trapezoidal rule. 8

10.7 Tasks (compute one trapezoid each) and the communication (send area to be summed). 9

10.8 Each rank integrates the sub-interval $[\text{local_a}, \text{local_b}]$ of width $\text{local_n} \cdot h$. 10

10.9 A tree-structured global sum: pairs combine in parallel at each level. 11

10.10 An alternative tree: rank r receives from $r + p/2$, then $r + p/4, \ldots$ 11

10.11 One way to implement Allreduce: a global sum followed by distribution of the result ($2 \log_2 p$ steps). 13

10.12 Butterfly-structured global sum: processes *exchange* partial results, so after $\log_2 p$ steps every process has the total. 14

10.13 A tree-structured broadcast from process 0: the number of holders doubles each step. 14

10.14 Owner of each of the 12 vector components under the three distributions. 15

10.15 The four basic collective patterns. 18

11.1 Two processes, two resources, one cycle. 20

11.2 Resource-allocation graphs: circles are processes, squares are resources. 22

11.3 (e)–(j): after step 6, C requests R and the cycle A→S→B→T→C→R→A closes: **deadlock**. 22

11.4 The four strategies and their techniques. 23

11.5 Detection with one resource of each type. 24

11.6 The four data structures of the multi-resource detection algorithm. 25

11.7 (l)–(q): by delaying B, no cycle ever forms. In (o) C waits for R; when A releases R (p), C gets it (q). 26

11.8 Two-process resource trajectories. Horizontal moves = A runs, vertical moves = B runs. Hatched regions are impossible (both would hold the same device). Once the trajectory enters the box $[I_1, I_2] \times [I_5, I_6]$ it is doomed to hit the forbidden regions. . 27

11.9 Spooling: the printer is never shared, so it can never be part of a cycle. 30

11.10 With numbered resources, requests only ever point "uphill", so a cycle cannot close. 30

12.1 A combining tree: threads enter at the leaves and climb towards the counter at the root. . . . 35
12.2 Combining as a parallel prefix (scan): sums flow up; offsets flow down so each leaf learns the value the counter had "before" its own increment (after J. Demmel, CS267). 36
12.3 Wait-free ⊂ lock-free ⊂ obstruction-free. . . . 37
12.4 A failed CaS means someone else succeeded — so the system as a whole always makes progress. . . . 38
12.5 A linked queue with a sentinel node. `enq()` and `deq()` work at **different ends** of the object. . . . 38
12.6 Steps of a successful enqueue. . . . 39
12.7 Steps of a successful dequeue: the old first node becomes the new sentinel. . . . 40
12.8 A monitor lock with a condition: `await()` releases the lock and moves the thread to the waiting room. . . . 41
12.9 Lost wake-up: two items were enqueued but only one waiter was ever woken. . . . 42
12.10 Splitting `permits` so the two ends rarely touch the same memory. . . . 42
12.11 Lock-free enqueue: CAS the `next` pointer, then CAS `tail`. . . . 43
12.12 Lock-free dequeue: read the value of `head.next`, then CAS `head` forward. . . . 44

13.1 Foster's methodology (after Dr. Dobb's "Designing Parallel Algorithms"). Partitioning and communication focus on **concurrency and scalability**; agglomeration and mapping focus on **locality** and other performance issues. . . . 47
13.2 Two ways to partition (the climate-model example is the classic illustration of functional decomposition). . . . 48
13.3 Dependence arrows decide whether a loop is parallel. . . . 49
13.4 The fork–join model: the master thread repeatedly forks a team of tasks and joins them (after Wikipedia, *Fork–join model*). . . . 50
13.5 Divide and conquer for Fibonacci: each `spawn` (purple) may run on another thread; `sync` is the join (Cormen et al., *Introduction to Algorithms*, 3rd ed.). . . . 51
13.6 Producer/consumer through a shared queue. . . . 51
13.7 A three-stage pipeline processing files f0–f4. . . . 52
13.8 Asynchronous agents: no central control; events trigger further events. . . . 52
13.9 A task queue feeding a fixed pool of worker threads. . . . 54
13.10 Work stealing: an idle thread takes work from the far end of a busy thread's deque (after karlsenchoi.blogspot.com). . . . 54

14.1 MapReduce as a computation graph: many independent maps feed one (logical) reduce. . . . 58
14.2 MapReduce word count: split, map to (word, 1), shuffle by key, reduce by summing (after xiaochongzhang.me). . . . 59
14.3 The MapReduce data model: maps emit (key, value) pairs; the framework sorts by key and hands each key's values to a reducer. . . . 60

14.4 Execution on a cluster: the master farms tasks out to workers and collects the results (after cbsolution.net). 61

14.5 MapReduce execution overview (redrawn from Dean & Ghemawat, OSDI 2004). . 61

14.6 A combiner is a "mini-reduce" run locally on each mapper's output. 62

14.7 Building an inverted index (term $\rightarrow$ documents). 63

LECTURE 10

Distributed Memory Programming with MPI

Overview

Moving from shared memory to **distributed memory**: every process has its own private memory, and the only way to share data is to **send messages**. This lecture introduces the **Message Passing Interface (MPI)** — the six "golden" functions, communicators, the SPMD style, point-to-point `MPI_Send`/`MPI_Recv` and message matching — then parallelises the **trapezoidal rule**, and finishes with **collective communication**: tree and butterfly reductions, `MPI_Reduce`, `MPI_Allreduce`, `MPI_Bcast`, data partitioning, `MPI_Scatter`, `MPI_Gather` and `MPI_Allgather`. Slides extended from *An Introduction to Parallel Programming* by Peter Pacheco.

10.1 Distributed Memory Systems

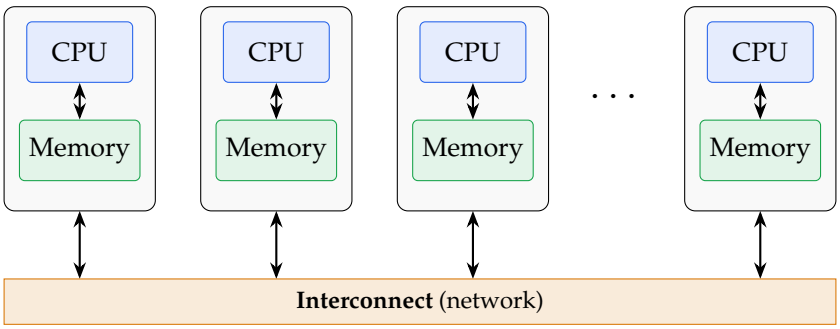


Figure 10.1: A distributed-memory system: each CPU can directly access only its own memory; everything else travels over the interconnect.

What changes compared with Lecture 4

In a **shared-memory** system all cores read and write the same address space, so threads communicate *implicitly*. In a **distributed-memory** system each node has **private memory**; a process cannot read another process's variables. Communication is **explicit**: one process **sends** a message and another **receives** it.
We develop programs for these systems using **MPI — the Message Passing Interface**: a **set of libraries** that can be called from **C, C++ and Fortran**.

	Shared memory (Pthreads/OpenMP)	Distributed memory (MPI)
Unit of execution	threads in one process	separate **processes**, possibly on different machines
Data sharing	any thread reads/writes shared variables	only by **explicit messages**
Main hazard	races on shared data	mismatched sends/receives, deadlock, communication cost
Scales to	one node (tens of cores)	**thousands** of nodes

10.2 Why MPI?

Reasons

- A **standardized and portable** message-passing system.
- **One of the oldest** parallel libraries, with **wide-spread adoption**.
- **Minimal requirements on the underlying hardware**.
- **Explicit parallelization**: the programmer decides what runs where.
- **Achieves high performance** and **scales to a large number of processors**.
- . . . but it is **intellectually demanding** — every data movement must be designed by hand.

10.3 Our First MPI Program

```c
#include <stdio.h>
#include <string.h>   /* For strlen             */
#include <mpi.h>      /* For MPI functions, etc */

const int MAX_STRING = 100;

int main(void) {
   char  greeting[MAX_STRING];
   int   comm_sz;   /* Number of processes */
   int   my_rank;   /* My process rank     */

   MPI_Init(NULL, NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   if (my_rank != 0) {
      sprintf(greeting, "Greetings from process %d of %d!",
            my_rank, comm_sz);
      MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
            MPI_COMM_WORLD);
   } else {
      printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
      for (int q = 1; q < comm_sz; q++) {
         MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
            0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
         printf("%s\n", greeting);
      }
```

```
28      }
29
30      MPI_Finalize();
31      return 0;
32  }   /* main */
```

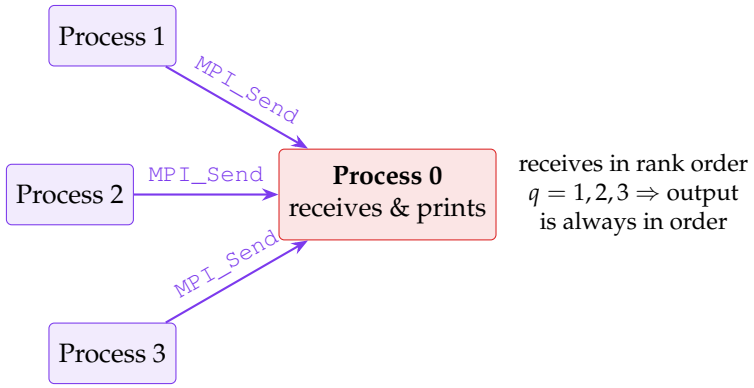


Figure 10.2: Hello world: ranks $1..p-1$ send a greeting; rank 0 receives them in rank order.

10.3.1 Compilation

Compile with the wrapper script

```
mpicc  -g  -Wall  -o mpi_hello  mpi_hello.c
```

`mpicc`	**wrapper script** around the C compiler (adds MPI include paths and libraries)
`-g`	produce debugging information
`-Wall`	turn on all warnings
`-o mpi_hello`	create this executable file name (as opposed to the default `a.out`)
`mpi_hello.c`	source file

10.3.2 Execution

Running with `mpiexec -n <number of processes> <executable>`

```
$ mpiexec -n 1 ./mpi_hello
Greetings from process 0 of 1 !
```

```
$ mpiexec -n 4 ./mpi_hello
Greetings from process 0 of 4 !
Greetings from process 1 of 4 !
Greetings from process 2 of 4 !
Greetings from process 3 of 4 !
```

Unlike the Pthreads "hello" of Lecture 4, the output order here is **deterministic**: process 0 does all the printing, and it receives from rank 1, then 2, then 3.

10.3.3 MPI Programs — Conventions

Naming

- Need to add the **`mpi.h`** header file.
- Identifiers defined by MPI **start with `MPI_`**.
- The **first letter after the underscore is uppercase** for function names and MPI-defined types (`MPI_Send`, `MPI_Comm`); constants are all caps (`MPI_COMM_WORLD`). This helps avoid confusion.

10.4 The Six Golden MPI Functions

Function	Purpose
`MPI_Init`	Initializes MPI
`MPI_Finalize`	Terminates MPI
`MPI_Comm_size`	Determines the number of processes
`MPI_Comm_rank`	Determines the label (rank) of the calling process
`MPI_Send`	Sends a message
`MPI_Recv`	Receives a message

Six is enough

Any MPI program *can* be written with just these six functions. Everything else (collectives, non-blocking calls, derived types) exists for convenience and performance.

10.4.1 `MPI_Init` and `MPI_Finalize`

```
int MPI_Init(
      int*      argc_p   /* in/out */,
      char***   argv_p   /* in/out */);

int MPI_Finalize(void);
```

What they do

`MPI_Init` tells MPI to do all necessary setup — e.g. allocate storage for message buffers and decide the rank of each process. `argc_p` and `argv_p` are pointers to `argc` and `argv` in `main()` (or `NULL`). The function returns an **error code**.
`MPI_Finalize` tells MPI we are done, so it can clean up anything allocated for this program.

```
#include <mpi.h>
int main(int argc, char* argv[]) {
   ...
   /* No MPI calls before this */
   MPI_Init(&argc, &argv);
   ...
   MPI_Finalize();
```

```
    /* No MPI calls after this */
    ...
    return 0;
}
```

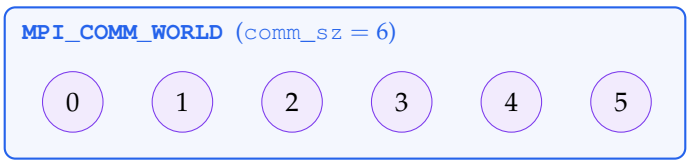
10.5 Communicators

Definition

A **communicator** is a **collection of processes that can send messages to each other**. Messages from other communicators are ignored. `MPI_Init` defines a communicator that consists of **all processes created when the program is started**, called **`MPI_COMM_WORLD`**.

```
int MPI_Comm_size(
      MPI_Comm  comm       /* in  */,
      int*      comm_sz_p  /* out */);   /* number of processes in the
   communicator */

int MPI_Comm_rank(
      MPI_Comm  comm       /* in  */,
      int*      my_rank_p  /* out */);   /* my rank (process making this
   call) */
```



ranks $0, 1, \ldots,$ `comm_sz` $- 1$

Figure 10.3: A communicator numbers its processes with ranks $0 \ldots p - 1$.

10.6 Single-Program Multiple-Data (SPMD)

The SPMD style

- We compile **one program**; every process runs a copy of it.
- Typically **process 0 does something different** — e.g. receives messages and prints them while the other processes do the work.
- The **`if-else`** construct on `my_rank` is what makes our program SPMD.
- We can run the program on **any number of processes**: 4, 8, 32, 1000, ...

10.7 Point-to-Point Communication

10.7.1 `MPI_Send`

```
int MPI_Send(
      void*         msg_buf_p    /* in */,
```

```
      int           msg_size     /* in */,
      MPI_Datatype  msg_type     /* in */,
      int           dest         /* in */,
      int           tag          /* in */,
      MPI_Comm      communicator /* in */);
```

Arguments

- `msg_buf_p`, `msg_size`, `msg_type` — determine the **content** of the message (a pointer, a count of elements, and their type).
- `dest` — the **destination process's rank**.
- `tag` — a non-negative integer used to **distinguish messages that are otherwise identical** (e.g. "data to print" vs "data to compute with").
- `communicator` — the communicator both processes belong to.

10.7.2 Data Types

MPI datatype	C datatype	MPI datatype	C datatype
`MPI_CHAR`	`signed char`	`MPI_UNSIGNED_SHORT`	`unsigned short int`
`MPI_SHORT`	`signed short int`	`MPI_UNSIGNED`	`unsigned int`
`MPI_INT`	`signed int`	`MPI_UNSIGNED_LONG`	`unsigned long int`
`MPI_LONG`	`signed long int`	`MPI_FLOAT`	`float`
`MPI_LONG_LONG`	`signed long long int`	`MPI_DOUBLE`	`double`
`MPI_UNSIGNED_CHAR`	`unsigned char`	`MPI_LONG_DOUBLE`	`long double`
		`MPI_BYTE`, `MPI_PACKED`	(raw bytes / packed data)

10.7.3 `MPI_Recv`

```
int MPI_Recv(
      void*         msg_buf_p    /* out */,
      int           buf_size     /* in  */,
      MPI_Datatype  buf_type     /* in  */,
      int           source       /* in  */,
      int           tag          /* in  */,
      MPI_Comm      communicator /* in  */,
      MPI_Status*   status_p     /* out */);
```

Wildcards

Pass **`MPI_ANY_SOURCE`** as `source` to receive messages from any sender **in order of arrival**, and **`MPI_ANY_TAG`** as `tag` to accept any tag. Pass `MPI_STATUS_IGNORE` if you don't need the status.

10.7.4 Message Matching

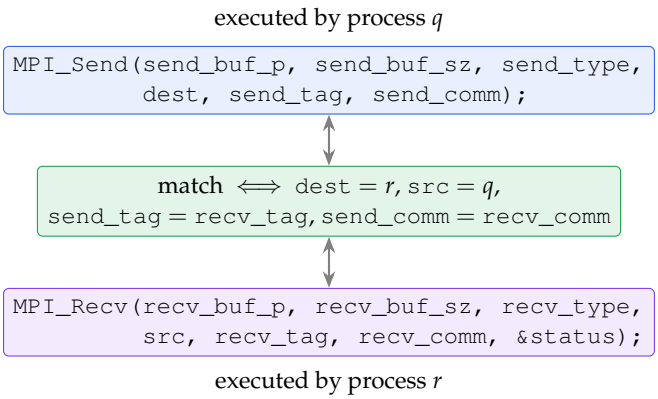


Figure 10.4: A message sent by q is received by r only if destination, source, tag and communicator all agree.

Also required

For the receive to succeed the buffers must be compatible: `recv_type` = `send_type` and `recv_buf_sz` $\geq$ `send_buf_sz`.

10.7.5 Receiving Messages Without Knowing Everything

A receiver can get a message without knowing...

...the **amount of data** in the message, the **sender** of the message (with `MPI_ANY_SOURCE`), or the **tag** of the message (with `MPI_ANY_TAG`). How can those be found out?

Answer: the `MPI_Status` object

The `status_p` argument of `MPI_Recv` is filled in with at least the fields `MPI_SOURCE`, `MPI_TAG` and `MPI_ERROR`. The **amount** of data received is obtained with:

```
int MPI_Get_count(
      MPI_Status*   status_p  /* in  */,
      MPI_Datatype  type      /* in  */,
      int*          count_p   /* out */);

MPI_Status status;
MPI_Recv(buf, MAX, MPI_DOUBLE, MPI_ANY_SOURCE, MPI_ANY_TAG, comm, &
    status);
int sender = status.MPI_SOURCE, tag = status.MPI_TAG, count;
MPI_Get_count(&status, MPI_DOUBLE, &count);
```

10.7.6 Issues with Send and Receive

Know your implementation — don't make assumptions

- The **exact behaviour is determined by the MPI implementation**.
- `MPI_Send` may behave differently with regard to **buffer size, cutoffs and blocking**. With a **cutoff**: if message size $<$ cutoff the message is **buffered** and `MPI_Send` returns; if message size $\geq$ cutoff, **`MPI_Send` will block** until a matching receive starts.
- **`MPI_Recv` always blocks** until a matching message is received.
- MPI **preserves message ordering from a given sender** (on the same communicator and tag): messages are **non-overtaking**.

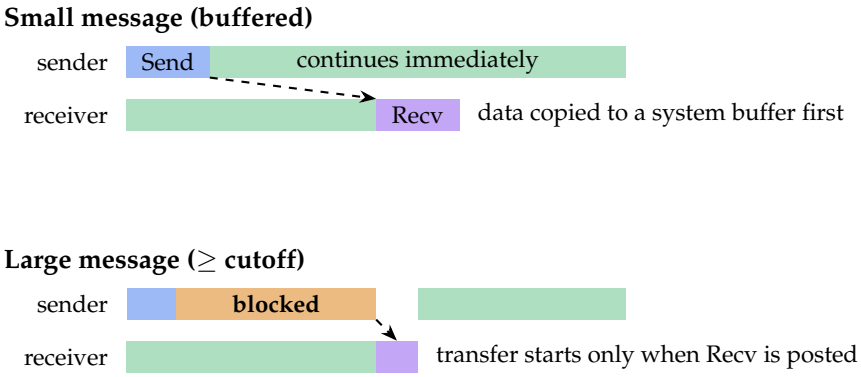


Figure 10.5: Whether `MPI_Send` returns early depends on the message size and the implementation.

Why this matters

A program that *happens* to work because small messages are buffered can **hang** when message sizes grow (both processes block in `MPI_Send`, each waiting for the other's receive). Never rely on buffering for correctness — see Lecture 11 on deadlocks.

10.8 Case Study: The Trapezoidal Rule in MPI

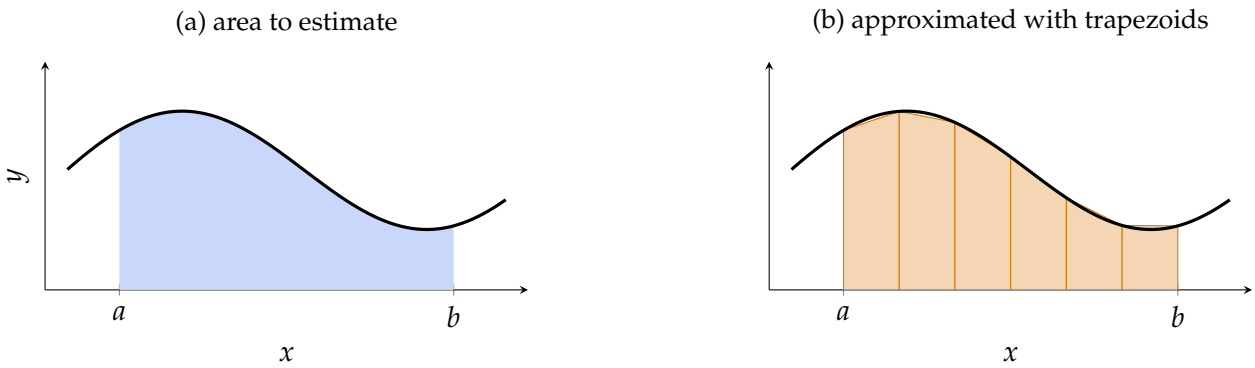


Figure 10.6: The trapezoidal rule.

The rule

$$\text{Area of one trapezoid} = \frac{h}{2}\left[f(x_i) + f(x_{i+1})\right], \qquad h = \frac{b-a}{n}$$

$$x_0 = a,\ x_1 = a+h,\ x_2 = a+2h,\ \ldots,\ x_{n-1} = a+(n-1)h,\ x_n = b$$

$$\text{Sum of trapezoid areas} = h\left[\frac{f(x_0)}{2} + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + \frac{f(x_n)}{2}\right]$$

10.8.1 Serial Pseudo-code

```
/* Input: a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

10.8.2 Parallelising: Tasks and Communication

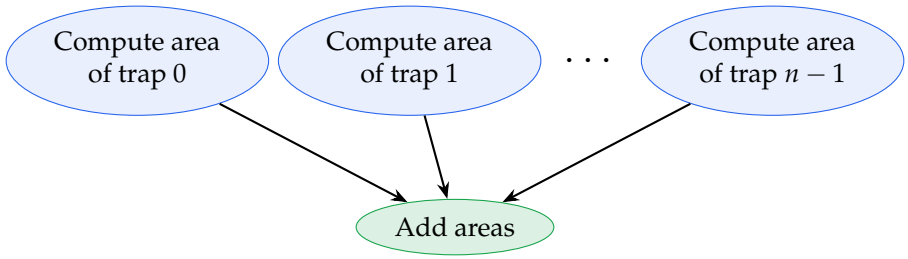


Figure 10.7: Tasks (compute one trapezoid each) and the communication (send area to be summed).

Agglomeration

There are far more trapezoids (n) than processes (p), so we **group** them: each process gets a contiguous block of local_n $= n/p$ trapezoids, computes their sum, and the partial sums are added on process 0.

10.8.3 Parallel Pseudo-code

```
1  Get a, b, n;
2  h = (b-a)/n;
3  local_n = n/comm_sz;
4  local_a = a + my_rank*local_n*h;
5  local_b = local_a + local_n*h;
6  local_integral = Trap(local_a, local_b, local_n, h);
7  if (my_rank != 0)
8     Send local_integral to process 0;
9  else /* my_rank == 0 */
10    total_integral = local_integral;
11    for (proc = 1; proc < comm_sz; proc++) {
12       Receive local_integral from proc;
13       total_integral += local_integral;
14    }
15 if (my_rank == 0)
```

```
16    print result;
```



Figure 10.8: Each rank integrates the sub-interval [local_a, local_b] of width local_n · h.

10.8.4 First Version

```
int main(void) {
   int my_rank, comm_sz, n = 1024, local_n;
   double a = 0.0, b = 3.0, h, local_a, local_b;
   double local_int, total_int;
   int source;

   MPI_Init(NULL, NULL);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

   h = (b-a)/n;           /* h is the same for all processes */
   local_n = n/comm_sz;   /* So is the number of trapezoids  */

   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   local_int = Trap(local_a, local_b, local_n, h);

   if (my_rank != 0) {
      MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
            MPI_COMM_WORLD);
   } else {
      total_int = local_int;
      for (source = 1; source < comm_sz; source++) {
         MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
               MPI_COMM_WORLD, MPI_STATUS_IGNORE);
         total_int += local_int;
      }
   }

   if (my_rank == 0) {
      printf("With n = %d trapezoids, our estimate\n", n);
      printf("of the integral from %f to %f = %.15e\n",
          a, b, total_int);
   }
   MPI_Finalize();
   return 0;
} /*  main  */
```

```
double Trap(
      double left_endpt  /* in */,
      double right_endpt /* in */,
      int    trap_count  /* in */,
      double base_len    /* in */) {
   double estimate, x;
   int i;

   estimate = (f(left_endpt) + f(right_endpt))/2.0;
```

```
10    for (i = 1; i <= trap_count-1; i++) {
11       x = left_endpt + i*base_len;
12       estimate += f(x);
13    }
14    estimate = estimate*base_len;
15
16    return estimate;
17 } /*  Trap  */
```

The weakness of version 1

Process 0 does $p-1$ receives and additions **one after another**, while every other process sits idle. The global sum takes $p-1$ communication steps — it does not scale. Collective communication fixes this.

10.9 Collective Communication

10.9.1 Tree-Structured Global Sum

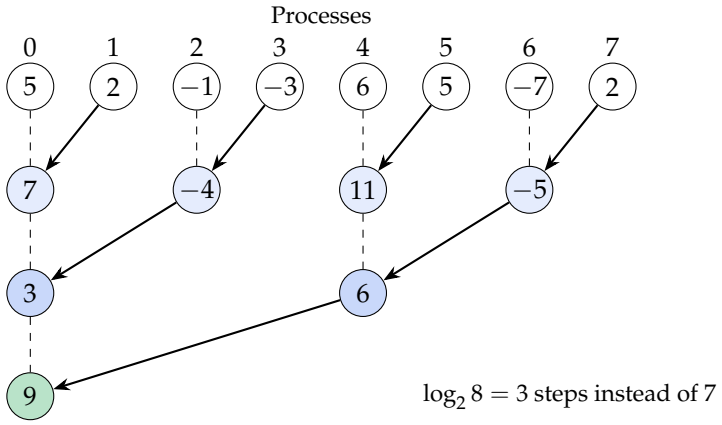


Figure 10.9: A tree-structured global sum: pairs combine in parallel at each level.

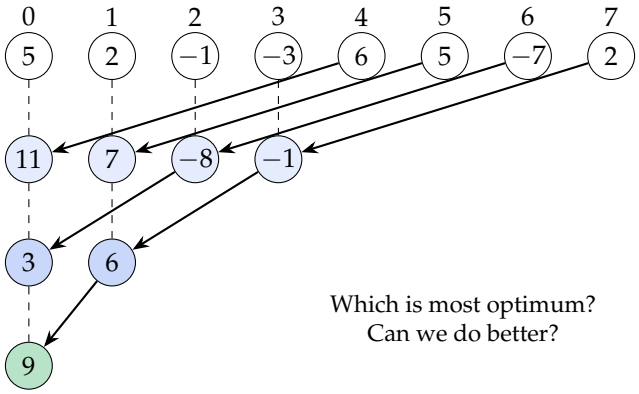


Figure 10.10: An alternative tree: rank r receives from $r + p/2$, then $r + p/4, \ldots$

Which tree is optimal?

Both need $\log_2 p$ communication steps, so on paper they are equal; which is faster in practice depends on the **network topology** (which pairs of nodes are physically close). That is exactly why MPI provides **built-in collectives**: the library implementer chooses the best structure for the machine, so you don't have to.

10.9.2 `MPI_Reduce`

```
int MPI_Reduce(
      void*          input_data_p    /* in  */,
      void*          output_data_p   /* out */,
      int            count           /* in  */,
      MPI_Datatype   datatype        /* in  */,
      MPI_Op         operator        /* in  */,
      int            dest_process    /* in  */,
      MPI_Comm       comm            /* in  */);
```

Using it

Replace the whole `if/else` send/receive block of version 1 with a single call:

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

With `count` > 1 it reduces arrays **element-wise**:

```
double local_x[N], sum[N];
...
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Operator	Meaning	Operator	Meaning
`MPI_MAX`	Maximum	`MPI_LOR`	Logical or
`MPI_MIN`	Minimum	`MPI_BOR`	Bitwise or
`MPI_SUM`	Sum	`MPI_LXOR`	Logical exclusive or
`MPI_PROD`	Product	`MPI_BXOR`	Bitwise exclusive or
`MPI_LAND`	Logical and	`MPI_MAXLOC`	Maximum and location of maximum
`MPI_BAND`	Bitwise and	`MPI_MINLOC`	Minimum and location of minimum

10.9.3 Collective vs Point-to-Point Communication

Rules for collectives

- **All processes in the communicator must call the same collective function.** E.g. a program that attempts to match a call to `MPI_Reduce` on one process with a call to `MPI_Recv` on another is erroneous — it will **hang or crash**.
- The **arguments** passed by each process must be **"compatible"**. E.g. if one process passes 0 as `dest_process` and another passes 1, the outcome of `MPI_Reduce` is erroneous and the program is likely to hang or crash.
- The `output_data_p` argument is **only used on `dest_process`**. However, all processes must still pass an actual argument for it, even if it's just `NULL`.

	Point-to-point	Collective
Participants	exactly 2 processes	**all** processes of the communicator
Matched by	**tags** and communicators	communicator and the **order in which they're called** (no tags)
Example	`MPI_Send` / `MPI_Recv`	`MPI_Reduce`, `MPI_Bcast`, ...

10.9.4 `MPI_Allreduce`

When to use it

Useful when **all processes need the result** of a global sum to complete some larger computation (e.g. a norm used to normalise every process's data).

```
int MPI_Allreduce(
      void*          input_data_p    /* in  */,
      void*          output_data_p   /* out */,
      int            count           /* in  */,
      MPI_Datatype   datatype        /* in  */,
      MPI_Op         operator        /* in  */,
      MPI_Comm       comm            /* in  */);    /* note: no dest_process */
```

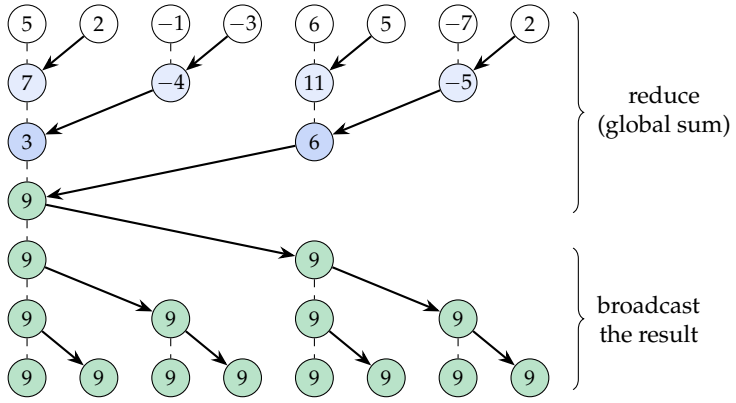


Figure 10.11: One way to implement Allreduce: a global sum followed by distribution of the result ($2 \log_2 p$ steps).

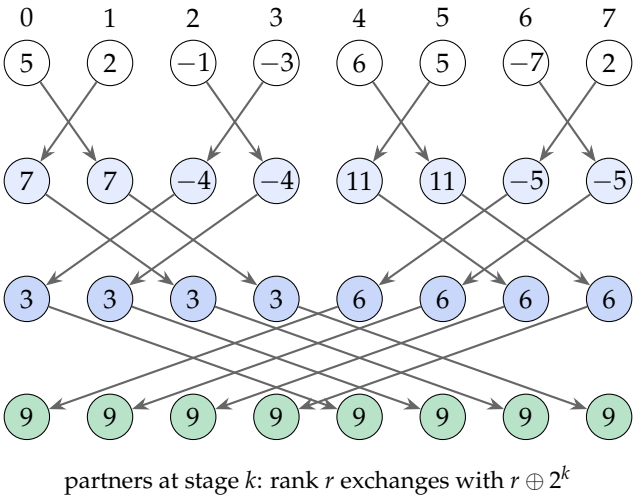


Figure 10.12: Butterfly-structured global sum: processes *exchange* partial results, so after $\log_2 p$ steps every process has the total.

Butterfly vs reduce-then-broadcast

The butterfly finishes in $\log_2 p$ steps (not $2\log_2 p$), because every process is both sending and receiving at every stage — no one sits idle while the result travels back down the tree.

10.9.5 Broadcast

Definition

Data belonging to a **single process is sent to all of the processes** in the communicator.

```
int MPI_Bcast(
      void*          data_p        /* in/out */,
      int            count         /* in     */,
      MPI_Datatype   datatype      /* in     */,
      int            source_proc   /* in     */,
      MPI_Comm       comm          /* in     */);
```

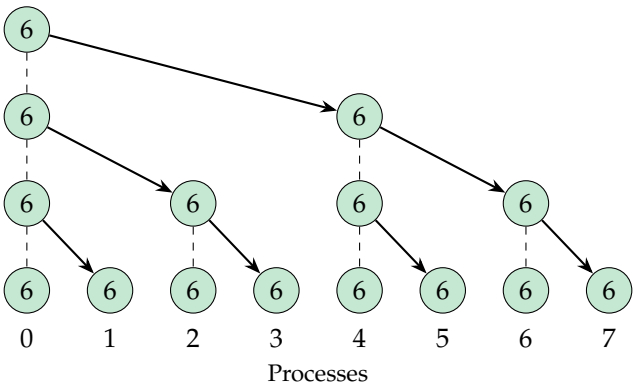


Figure 10.13: A tree-structured broadcast from process 0: the number of holders doubles each step.

Typical use: reading input once

```
if (my_rank == 0) {
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
}
MPI_Bcast(&a, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(&b, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(&n, 1, MPI_INT,    0, MPI_COMM_WORLD);
```

Only process 0 can read `stdin` reliably, so it reads and broadcasts.

10.10 Data Distributions

10.10.1 Example: Computing a Vector Sum

$$\mathbf{x} + \mathbf{y} = (x_0, \ldots, x_{n-1}) + (y_0, \ldots, y_{n-1}) = (x_0 + y_0, \ldots, x_{n-1} + y_{n-1}) = (z_0, \ldots, z_{n-1}) = \mathbf{z}$$

```
void Vector_sum(double x[], double y[], double z[], int n) {
   int i;
   for (i = 0; i < n; i++)
      z[i] = x[i] + y[i];
}  /* Vector_sum */
```

10.10.2 Partitioning Options

Process	**Block**				**Cyclic**				**Block-cyclic** (blocksize 2)			
0	0	1	2	3	0	3	6	9	0	1	6	7
1	4	5	6	7	1	4	7	10	2	3	8	9
2	8	9	10	11	2	5	8	11	4	5	10	11

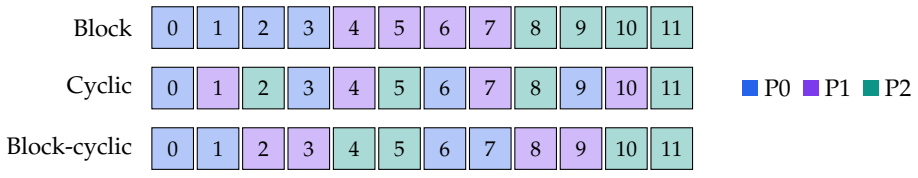


Figure 10.14: Owner of each of the 12 vector components under the three distributions.

The three options

- **Block partitioning** — assign **blocks of consecutive components** to each process.
- **Cyclic partitioning** — assign components in a **round-robin** fashion.
- **Block-cyclic partitioning** — use a **cyclic distribution of blocks** of components.

Cyclic and block-cyclic help with **load balance** when work per component varies (e.g. triangular matrices); block is simplest and keeps neighbours together.

10.10.3 Parallel Implementation

```
void Parallel_vector_sum(
      double  local_x[]  /* in  */,
      double  local_y[]  /* in  */,
      double  local_z[]  /* out */,
      int     local_n    /* in  */) {
   int local_i;

   for (local_i = 0; local_i < local_n; local_i++)
      local_z[local_i] = local_x[local_i] + local_y[local_i];
}  /* Parallel_vector_sum */
```

No communication at all

Once the vectors are distributed, each process adds **its own** block — a perfectly data-parallel step. The communication happens only when distributing the input (scatter) and collecting the output (gather).

10.10.4 `MPI_Scatter`

Purpose

Can be used in a function that **reads in an entire vector on process 0** but **only sends the needed components** to the other processes.

```
int MPI_Scatter(
      void*         send_buf_p  /* in  */,
      int           send_count  /* in  */,
      MPI_Datatype  send_type   /* in  */,
      void*         recv_buf_p  /* out */,
      int           recv_count  /* in  */,
      MPI_Datatype  recv_type   /* in  */,
      int           src_proc    /* in  */,
      MPI_Comm      comm        /* in  */);
```

```
void Read_vector(
      double    local_a[]   /* out */,
      int       local_n     /* in  */,
      int       n           /* in  */,
      char      vec_name[]  /* in  */,
      int       my_rank     /* in  */,
      MPI_Comm  comm        /* in  */) {

   double* a = NULL;
   int i;

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      printf("Enter the vector %s\n", vec_name);
      for (i = 0; i < n; i++)
         scanf("%lf", &a[i]);
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
            0, comm);
      free(a);
   } else {
```

```
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
            0, comm);
   }
}  /* Read_vector */
```

send_count is per process

`send_count` is the number of elements sent **to each process**, not the total. `MPI_Scatter` uses a **block** distribution and assumes n is divisible by `comm_sz`.

10.10.5 MPI_Gather

Purpose

Collect all components of a vector onto process 0, so that process 0 can then process (e.g. print) all of them.

```
int MPI_Gather(
      void*          send_buf_p  /* in  */,
      int            send_count  /* in  */,
      MPI_Datatype   send_type   /* in  */,
      void*          recv_buf_p  /* out */,
      int            recv_count  /* in  */,
      MPI_Datatype   recv_type   /* in  */,
      int            dest_proc   /* in  */,
      MPI_Comm       comm        /* in  */);
```

10.10.6 MPI_Allgather

Purpose

Concatenates the contents of each process's `send_buf_p` and stores the result in **each** process's `recv_buf_p`. As usual, `recv_count` is the amount of data being received **from each process**.

```
int MPI_Allgather(
      void*          send_buf_p  /* in  */,
      int            send_count  /* in  */,
      MPI_Datatype   send_type   /* in  */,
      void*          recv_buf_p  /* out */,
      int            recv_count  /* in  */,
      MPI_Datatype   recv_type   /* in  */,
      MPI_Comm       comm        /* in  */);
```

10.11 Summary of Collectives

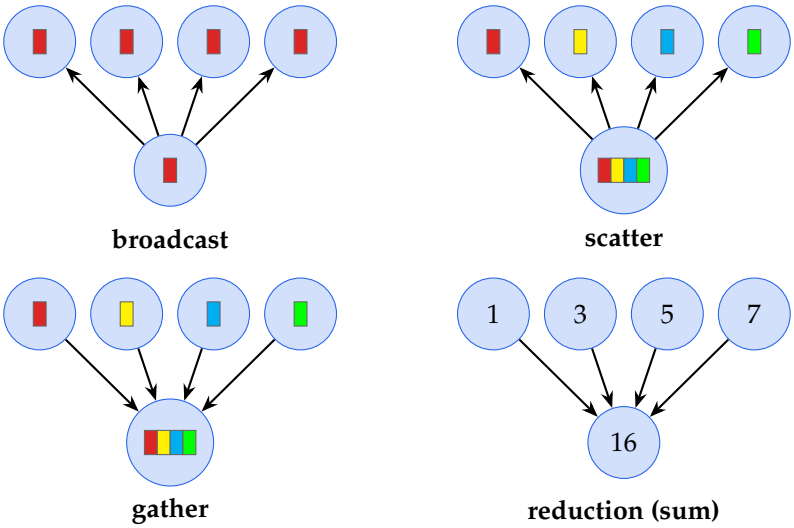


Figure 10.15: The four basic collective patterns (after the LLNL MPI tutorial, `https://computing.llnl.gov/tutorials/mpi/`).

Collective	What happens	Result lives on
`MPI_Bcast`	one process's data copied to all	all
`MPI_Scatter`	root's array split into blocks, one per process	each (one block)
`MPI_Gather`	every process's block concatenated on root	root
`MPI_Allgather`	gather, then everyone gets the full array	all
`MPI_Reduce`	combine with an operator (sum, max, ...)	root
`MPI_Allreduce`	reduce, then everyone gets the result	all

10.12 Summary

Lecture Summary

- In a **distributed-memory** system each process has **private memory**; data moves only through **explicit messages**. **MPI** is the standard, portable library for this (C, C++, Fortran).
- Compile with **`mpicc`**, run with **`mpiexec -n <p> ./prog`**.
- The **six golden functions**: `MPI_Init`, `MPI_Finalize`, `MPI_Comm_size`, `MPI_Comm_rank`, `MPI_Send`, `MPI_Recv`.
- A **communicator** is a group of processes that can message each other; `MPI_COMM_WORLD` contains them all. Ranks run $0 \ldots p-1$.
- Programs are **SPMD**: one program, branching on `my_rank`.
- A send from q matches a receive on r when **dest, source, tag and communicator** agree. Use `MPI_ANY_SOURCE`/`MPI_ANY_TAG`, `MPI_Status` and `MPI_Get_count` to discover what arrived.
- `MPI_Recv` **always blocks**; `MPI_Send` may buffer or block depending on size and

implementation — **don't assume**. Messages from one sender are **non-overtaking**.

- **Trapezoidal rule**: each rank integrates a block; partial sums are combined.
- **Collectives** (no tags; every process must call them with compatible arguments): tree/butterfly **reductions** in $\log_2 p$ steps, `MPI_Reduce`, `MPI_Allreduce`, `MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, `MPI_Allgather`.
- **Data distributions**: block, cyclic, block-cyclic.

Self-check Questions

1. Contrast shared-memory and distributed-memory programming in terms of how processes share data.
2. List the six golden MPI functions and the purpose of each.
3. What is a communicator, and what does `MPI_COMM_WORLD` contain?
4. State every condition required for an `MPI_Send` on process q to match an `MPI_Recv` on process r.
5. How does a receiver that used `MPI_ANY_SOURCE` find out who sent the message and how long it was?
6. Why can a program that works for small messages hang for large ones?
7. In version 1 of the trapezoidal program, how many sequential receive steps does process 0 perform? How many does a tree-structured sum need for $p = 1024$?
8. Explain the difference between reduce-then-broadcast and a butterfly allreduce.
9. Give two rules that collective calls must obey that point-to-point calls do not.
10. For $n = 12$ and $p = 3$, list the components owned by process 1 under block, cyclic and block-cyclic (blocksize 2) distributions.
11. What do `MPI_Scatter`, `MPI_Gather` and `MPI_Allgather` do, and what does `send_count` mean in a scatter?

LECTURE 11

Deadlocks

Overview

A systematic treatment of **deadlock**: what resources are, the formal definition, the **four Coffman conditions**, and **resource-allocation graphs**. Then the four strategies for dealing with deadlock — **ignore it** (the ostrich algorithm), **detect and recover**, **avoid** it dynamically (resource trajectories, safe states, the **Banker's algorithm**), and **prevent** it by negating one of the four conditions. Slides extended from *Modern Operating Systems* by A. S. Tanenbaum.

11.1 Resources

What is a resource?

- Examples of computer resources: **printers, tape drives**, plotters, scanners, locks, database records, ...
- Processes need access to resources in a **reasonable order**.
- Suppose a process holds resource T and requests resource U, while at the same time another process holds U and requests T — **both are blocked and remain so**.

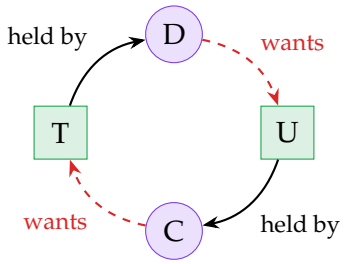


Figure 11.1: Two processes, two resources, one cycle.

Resources (cont.)

- Deadlocks occur when processes are granted **exclusive access** to devices/resources.
- The sequence of events required to use a resource is **request** → **use** → **release**.
- If a request is denied the process **must wait**: it may be **blocked**, or the request may **fail** with an error code.

11.1.1 Two Types of Resources

Type	Meaning	Example
Preemptable	can be taken away from a process with **no ill effects**	memory (can be swapped out), CPU
Nonpreemptable	will **cause the process to fail** if taken away	printer mid-job, CD burner

Why the distinction matters

Deadlocks that involve only preemptable resources can usually be resolved by reallocating them. The interesting deadlocks involve **nonpreemptable** resources.

11.2 Deadlocks

Formal definition

A set of processes is **deadlocked** if **each process in the set is waiting for an event that only another process in the set can cause**. Usually the event is the **release of a currently held resource**. None of the processes can **run**, **release resources**, or **be awakened**.

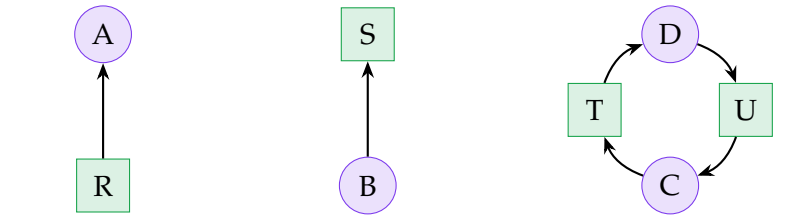
11.2.1 Four Conditions for Deadlock

	Condition	Meaning
1	**Mutual exclusion**	Each resource is assigned to 1 process or is available
2	**Hold and wait**	A process holding resources can request additional ones
3	**No preemption**	Previously granted resources cannot be forcibly taken away
4	**Circular wait**	There must be a circular chain of 2 or more processes, each waiting for a resource held by the next member of the chain

All four are necessary

All four conditions must hold for deadlock to occur. If any one is absent, deadlock is impossible — which is exactly what prevention exploits (§11.8).

11.3 Deadlock Modelling



(a) R assigned to A (b) B requests S (c) C and D deadlocked

Figure 11.2: Resource-allocation graphs: circles are processes, squares are resources.

Reading the graph

- Arrow **resource** → **process**: the resource is **assigned to** (held by) the process — (a) R is assigned to A.
- Arrow **process** → **resource**: the process is **requesting / waiting for** the resource — (b) B waits for S.
- A **cycle** means deadlock (for single-instance resources) — (c) C and D are deadlocked over T and U.

11.3.1 Example: How Deadlock Occurs

Process A	Process B	Process C
Request R	Request S	Request T
Request S	Request T	Request R
Release R	Release S	Release T
Release S	Release T	Release R

If the OS runs the requests in the interleaved order 1. A requests R, 2. B requests S, 3. C requests T, 4. A requests S, 5. B requests T, 6. C requests R:

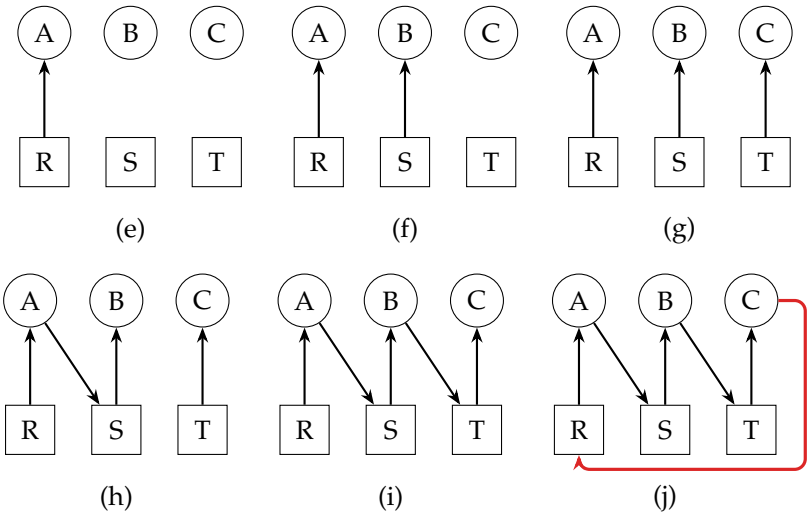


Figure 11.3: (e)–(j): after step 6, C requests R and the cycle A→S→B→T→C→R→A closes: **deadlock**.

11.4 Strategies for Dealing with Deadlocks

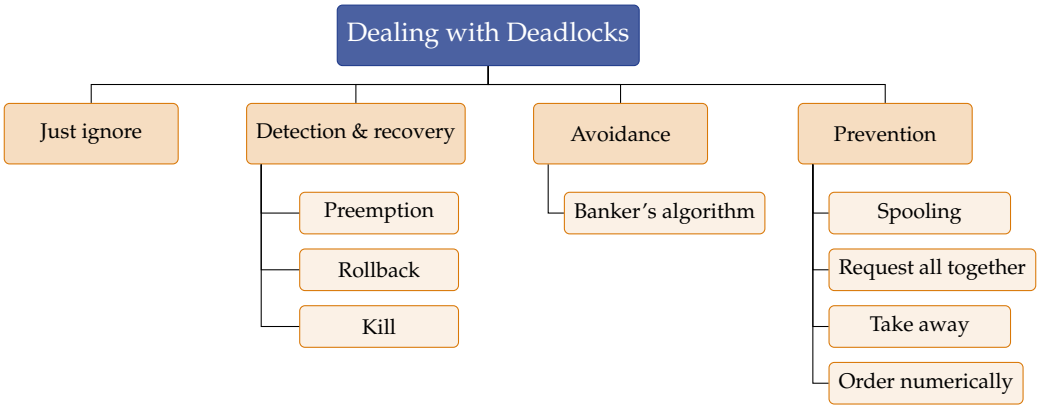


Figure 11.4: The four strategies and their techniques.

The four strategies

1. **Just ignore the problem** altogether.
2. **Detection and recovery**.
3. **Dynamic avoidance** — careful resource allocation.
4. **Prevention** — negating one of the 4 necessary conditions.

11.5 Strategy 1: The Ostrich Algorithm

Pretend there is no problem

Reasonable if **deadlocks occur very rarely** and the **cost of prevention is high**. **UNIX and Windows take this approach.** It is a trade-off between **convenience** and **correctness**.

Why it is not as silly as it sounds

If a deadlock happens once a year and the system crashes for other reasons once a week, paying a heavy price in performance or restrictions to eliminate deadlock is not worth it. Engineers "stick their heads in the sand" deliberately.

11.6 Strategy 2: Detection and Recovery

11.6.1 Detection with One Resource of Each Type

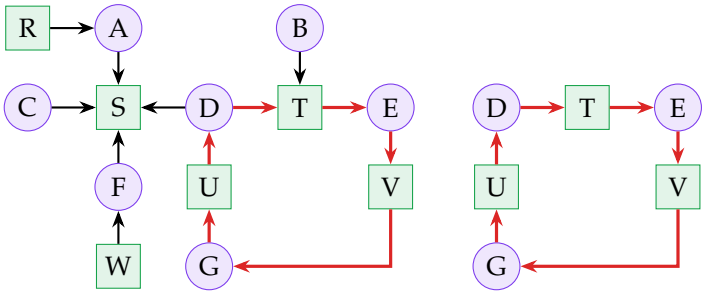


Figure 11.5: (a) A resource graph for 7 processes and 6 resources. (b) The cycle D→T→E→V→G→U→D: processes D, E and G are deadlocked.

Reading the system state (Tanenbaum's example)

A holds R, wants S; B holds nothing, wants T; C wants S; D holds U, wants S and T; E holds T, wants V; F holds W, wants S; G holds V, wants U. **Note resource ownership and requests**: a **cycle** can be found within the graph, denoting deadlock — **D, E and G are deadlocked**. A, C and F are **not** deadlocked: S is currently free, so it can be granted to any one of them, which then finishes and releases it for the next. (B waits for T, held by the deadlocked E, so B will also never run — it is stuck, though not part of the cycle.)

Algorithm

With exactly one instance of each resource, **deadlock** $\iff$ **the graph contains a cycle**. Detect it with any cycle-finding algorithm, e.g. a depth-first search from each node that reports a cycle when it revisits a node already on the current path. Cost: $O(\text{nodes} + \text{edges})$.

11.6.2 Detection with Multiple Resources of Each Type

Data structures needed by the detection algorithm

n processes, m resource classes.

- **Resources in existence** $E = (E_1, E_2, \ldots, E_m)$
- **Resources available** $A = (A_1, A_2, \ldots, A_m)$
- **Current allocation matrix** C ($n \times m$): row i is the current allocation to process i
- **Request matrix** R ($n \times m$): row i is what process i still needs

Invariant: $\sum_{i=1}^{n} C_{ij} + A_j = E_j$ for every resource class j.

Current allocation matrix

$$C = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nm} \end{bmatrix}$$

row n = current allocation to process n

Request matrix

$$R = \begin{bmatrix} R_{11} & R_{12} & \cdots & R_{1m} \\ R_{21} & R_{22} & \cdots & R_{2m} \\ \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & \cdots & R_{nm} \end{bmatrix}$$

row 2 = what process 2 needs

Figure 11.6: The four data structures of the multi-resource detection algorithm.

```
mark all processes unmarked
repeat
    find an unmarked process Pi whose request row Ri <= A   (component-wise)
    if found:
        A = A + Ci          // pretend Pi runs to completion and releases
   everything
        mark Pi
until no such process exists
any processes still unmarked are DEADLOCKED
```

Worked Example

Tape drives, plotters, scanners, CD-ROMs

$$E = (4\,2\,3\,1), \qquad A = (2\,1\,0\,0)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix} \qquad R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Step	Can run?	Why	New A
1	P_1: ✗	needs a CD-ROM, $A_4 = 0$	
	P_2: ✗	needs a scanner, $A_3 = 0$	
	P_3: ✓	$R_3 = (2100) \le (2100)$	$(2100) + (0120) = (2220)$
2	P_2: ✓	$(1010) \le (2220)$	$(2220) + (2001) = (4221)$
3	P_1: ✓	$(2001) \le (4221)$	$(4221) + (0010) = (4231) = E$

All processes can finish (order P_3, P_2, P_1): **no deadlock**. (If P_2 also wanted a CD-ROM, e.g. $R_2 = (1011)$, then after P_3 finished neither P_1 nor P_2 could proceed — both deadlocked.)

11.6.3 Recovery from Deadlock

Method	How
Preemption	Take a resource from some other process. Depends on the **nature of the resource** (often needs manual intervention).
Rollback	**Checkpoint** a process periodically; use this saved state; **restart the process** from a checkpoint if it is found deadlocked.
Killing processes	Kill one of the processes in the deadlock cycle so the others get its resources. Choose a process that **can be rerun from the beginning**.

11.7 Strategy 3: Deadlock Avoidance

The idea

Allow all four conditions in principle, but **grant each request only if it is safe**: the OS looks ahead and refuses (delays) any allocation that could lead to deadlock.

11.7.1 How Deadlock Can Be Avoided

Same three processes as before, but the OS chooses a different order: 1. A requests R, 2. C requests T, 3. A requests S, 4. C requests R, 5. A releases R, 6. A releases S — **B is suspended** (not granted S) until A is done.

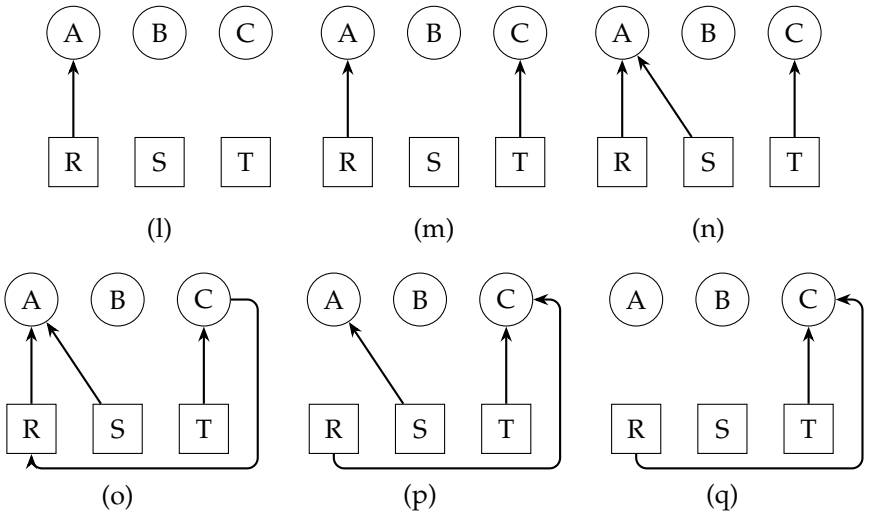


Figure 11.7: (l)–(q): by delaying B, no cycle ever forms. In (o) C waits for R; when A releases R (p), C gets it (q).

11.7.2 Resource Trajectories

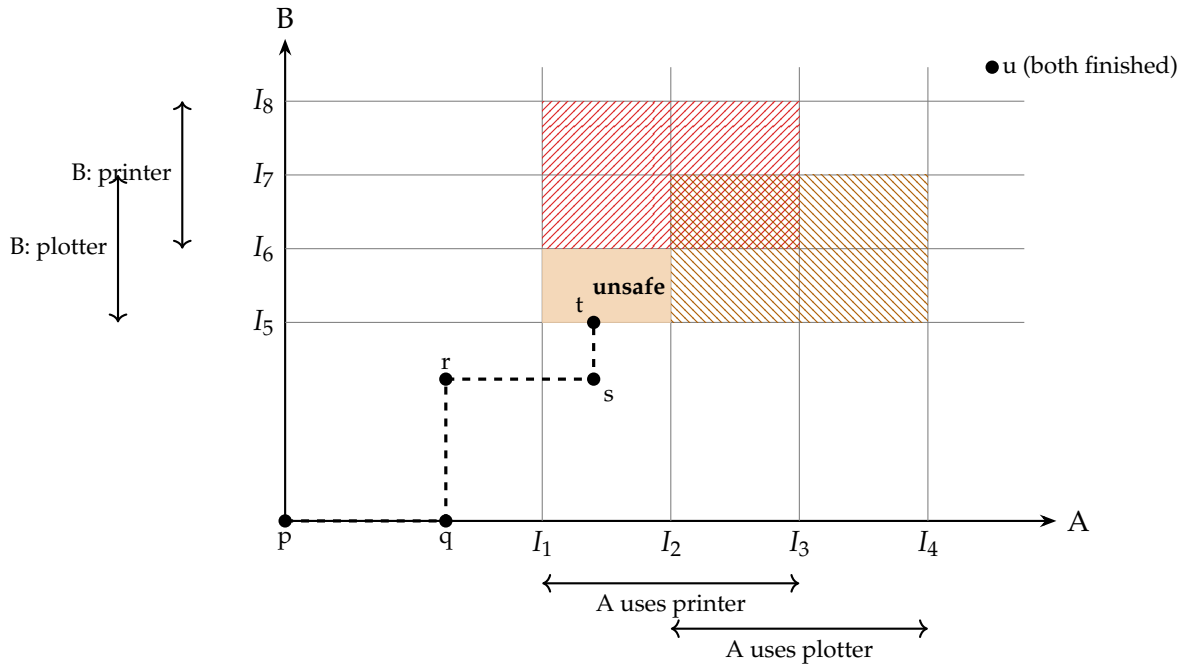


Figure 11.8: Two-process resource trajectories. Horizontal moves = A runs, vertical moves = B runs. Hatched regions are impossible (both would hold the same device). Once the trajectory enters the box $[I_1, I_2] \times [I_5, I_6]$ it is doomed to hit the forbidden regions.

How to read it

Each point is a joint state of the two processes' progress. A path only moves **right** (A executes) or **up** (B executes). Regions where both would hold the printer (or both the plotter) are **forbidden**. At t, if B is allowed to request the plotter, the system enters an **unsafe** region from which every path leads into a forbidden one — deadlock. The avoidance algorithm must therefore **suspend B at** t until A has released the printer (past I_3).

11.7.3 Safe and Unsafe States

Definition

A state is **safe** if there exists **some scheduling order** in which every process can run to completion, even if all of them suddenly request their **maximum** remaining resources. Otherwise it is **unsafe**. An unsafe state is **not** yet a deadlock — but the system can no longer *guarantee* that all processes will finish.

Demonstration that state (a) is safe (one resource type, 10 units)

(a)

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3 →

(b)

	Has	Max
A	3	9
B	**4**	4
C	2	7

Free: 1 →

(c)

	Has	Max
A	3	9
B	0	–
C	2	7

Free: 5 →

(d)

	Has	Max
A	3	9
B	0	–
C	**7**	7

Free: 0 →

(e)

	Has	Max
A	3	9
B	0	–
C	0	–

Free: 7

Give B its 2 more ⇒ B finishes and frees 4 (Free 5). Give C its 5 more ⇒ C finishes (Free 7). A needs $6 \leq 7$ ⇒ A finishes. Safe sequence: **B, C, A**.

Demonstration that state (b) is NOT safe

(a)

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

A gets 1 more →

(b)

	Has	Max
A	**4**	9
B	2	4
C	2	7

Free: 2

Only B can be guaranteed to finish (needs 2, frees 4 ⇒ Free 4). Now A needs 5 and C needs 5, but only 4 are free: **no process is guaranteed to finish**. Granting A's request moved the system from a safe to an unsafe state, so the Banker refuses it.

11.7.4 The Banker's Algorithm (Dijkstra)

Rule

Like a small-town banker granting lines of credit: when a request arrives, **pretend to grant it** and check whether the resulting state is **safe**. If yes, grant it; if no, **postpone** the request.

Banker's Algorithm for Multiple Resources

Resources assigned (C)

Proc	Tape	Plot	Scan	CD
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources still needed (R)

Proc	Tape	Plot	Scan	CD
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

$E = (6\,3\,4\,2)$ existing
$P = (5\,3\,2\,2)$ possessed
$A = (1\,0\,2\,0)$ available

```
1. Look for a row of R whose unmet needs are all <= A.
   If none exists, the state is unsafe (the system may eventually deadlock).
2. Assume that process requests all it needs and finishes:
   mark it terminated and add its row of C to A.
3. Repeat 1-2 until all processes are marked (safe) or none can proceed (
   unsafe).
```

Checking the state above

Step	Process	Need $\leq A$?	New A
0	—	—	(1020)
1	D	$(0010) \leq (1020)$ ✓	$+(1101) = (2121)$
2	A	$(1100) \leq (2121)$ ✓	$+(3011) = (5132)$
3	B	$(0112) \leq (5132)$ ✓	$+(0100) = (5232)$
4	C	$(3100) \leq (5232)$ ✓	$+(1110) = (6342)$
5	E	$(2110) \leq (6342)$ ✓	$+(0000) = (6342) = E$

Safe sequence **D, A, B, C, E** exists ⇒ the current state is **safe**.

Why the Banker's algorithm is rarely used in practice

Processes rarely know their **maximum needs** in advance; the **number of processes** changes dynamically; and resources can **vanish** (a device breaks). It is elegant theory, not a common OS mechanism.

11.8 Strategy 4: Deadlock Prevention

Idea

Make deadlock **structurally impossible** by ensuring that **at least one** of the four necessary conditions can never hold.

11.8.1 Attacking the Mutual Exclusion Condition

Spooling

- Some devices (such as a printer) can be **spooled**: processes write their output to a spool directory, and **only the printer daemon uses the printer** resource. Thus deadlock for the printer is eliminated.
- **Not all devices can be spooled** (and the spool disk itself can fill up and become contended).
- Principle: **avoid assigning resources when not absolutely necessary**; let **as few processes as possible** actually claim the resource.

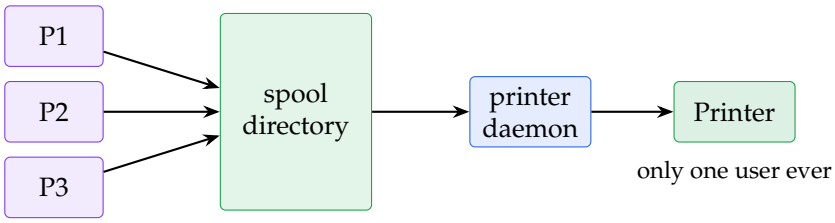


Figure 11.9: Spooling: the printer is never shared, so it can never be part of a cycle.

11.8.2 Attacking the Hold and Wait Condition

Request everything up front

Require processes to **request all resources before starting**; a process then never has to wait for what it needs. **Problems:** a process **may not know** its required resources at the start of the run; and it **ties up resources** other processes could be using. **Variation:** before requesting anything new, a process must **give up all resources** it holds, then request **all** it immediately needs.

11.8.3 Attacking the No Preemption Condition

Not a viable option

Consider a process given the printer, **halfway through its job**. Now forcibly take the printer away … !!?? The output is ruined. Preemption only works for resources whose state can be saved and restored (CPU, memory).

11.8.4 Attacking the Circular Wait Condition

Global ordering

Number all resources, and require that processes request resources only in **strictly increasing (or decreasing) order**. In other words, to request resource R_j, a process must first release all R_i such that $i \geq j$.

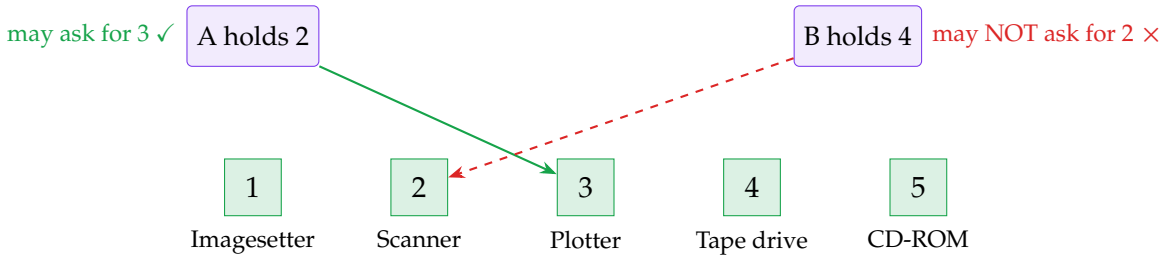


Figure 11.10: With numbered resources, requests only ever point "uphill", so a cycle cannot close.

Is this generally feasible?

It works well for locks inside a program (this is the lock-ordering rule from Lecture 6 and the dining-philosophers fix from Lecture 8). For general OS resources it is hard to find an ordering that suits every program, and the number of resources can be very large.

11.8.5 Summary of Prevention Approaches

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

11.9 Summary

Lecture Summary

- Resources are used by **request** → **use** → **release**; they are **preemptable** (no harm if taken) or **nonpreemptable** (process fails if taken).
- A set of processes is **deadlocked** if each waits for an event only another member can cause.
- **Four necessary conditions**: mutual exclusion, hold and wait, no preemption, circular wait.
- **Resource-allocation graphs**: resource→process = held, process→resource = requested; with single-instance resources a **cycle** $\iff$ **deadlock**.
- **Ostrich**: ignore it (UNIX, Windows) — convenience over correctness.
- **Detection** with multiple instances uses E, A, C, R; repeatedly "run" any process whose request fits in A and reclaim its allocation. **Recovery**: preemption, rollback, killing.
- **Avoidance**: only grant requests that keep the system in a **safe state**; resource trajectories visualise this; the **Banker's algorithm** checks safety.
- **Prevention**: spool (ME), request all up front (H&W), take away (NP — rarely viable), **order resources numerically** (CW — the practical one).

Self-check Questions

1. Give one example each of a preemptable and a nonpreemptable resource.
2. State the four conditions for deadlock. Why must all four hold?
3. Draw the resource-allocation graph for: A holds R and wants S; B holds S and wants R. Is there a deadlock?
4. Why is the ostrich algorithm a reasonable choice for general-purpose operating systems?
5. Run the multi-resource detection algorithm on $E = (4\,2\,3\,1)$, $A = (2\,1\,0\,0)$ with the C and R matrices of the worked example. In what order can the processes finish?
6. Define a safe state. Is every unsafe state a deadlock?
7. With one resource type and 10 units, is (A: has 4, max 9; B: has 2, max 4; C: has 2, max 7) safe? Justify.
8. Apply the Banker's algorithm to the five-process, four-resource example and give a safe sequence.

9. For each of the four conditions, give a prevention technique and a drawback.
10. Why is ordering resources numerically sufficient to prevent circular wait?

LECTURE 12

Concurrent Data Structures

Overview

How to build data structures that many threads can use at once *and* that perform well. Starting from the locked shared counter, the lecture covers **blocking techniques** (coarse- vs fine-grained locking, the **combining tree**), **non-blocking techniques** and their progress conditions (**wait-free, lock-free, obstruction-free**), **compare-and-swap**, and then two queues: a **bounded two-lock queue** (with `permits`, condition variables, the **lost wake-up** problem and a **split counter**) and the **lock-free (Michael–Scott) queue** built from CAS. Slides adapted from *The Art of Multiprocessor Programming* by Maurice Herlihy & Nir Shavit.

12.1 Motivation

Why concurrent data structures?

- Many/multi-core processors ⇒ **multiple threads working on shared data structures** (shared-memory multiprocessors). Threads **communicate and synchronize through data structures**.
- Locks, semaphores, mutexes and monitors satisfy **safety and liveness** properties…
- … **but they lack performance**: they make everything **serial**.

12.1.1 Example: `getAndIncrement()`

From the prime-number example of Lecture 1, with a shared counter:

```
public class Counter {
  private long value;

  public long getAndIncrement() {
    return value++;
  }
}
```

```
// value++ is really:
temp  = value;
value = value + 1;
return temp;
```

Solution: a synchronized block (mutual exclusion)

```
public long getAndIncrement() {
  Lock();
  temp  = value;
  value = temp + 1;
  Unlock();
  return temp;
}
```

12.1.2 Performance Problems with Locking

Two costs

- Code between `Lock()` and `Unlock()` is **sequential**. By **Amdahl's law**, even a small sequential fraction reduces performance.
- **Memory contention**: every processor needs to access the lock variable(s), giving **low performance on cache-coherent multiprocessors** (the lock's cache line bounces between cores).

12.2 Blocking Techniques

Technique	Example
Coarse-grained locking	locking an **entire** linked list
Fine-grained locking	locking **individual elements** of a linked list

What about our shared counter?

There is **only one element** to access, so fine-grained locking doesn't help. A possible solution: a **combining tree**.

12.2.1 The Combining Tree

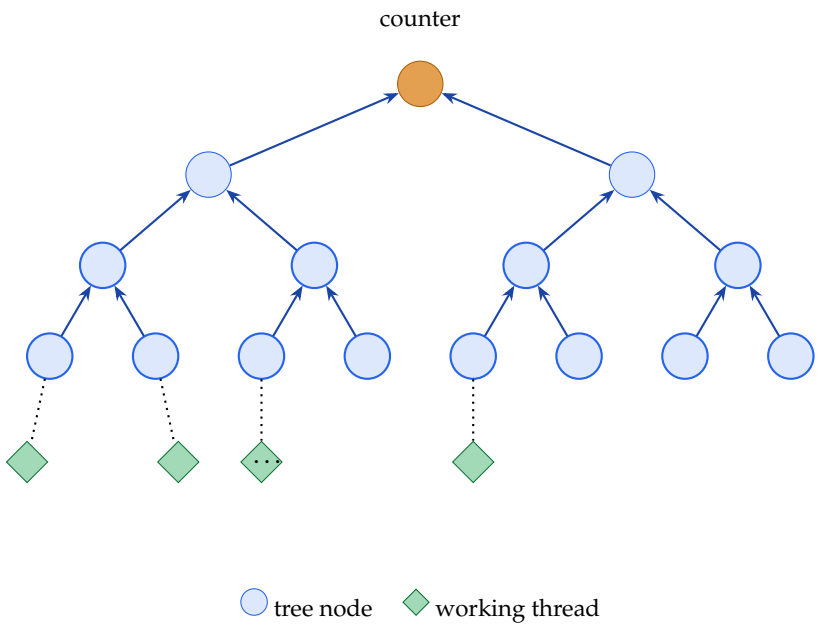


Figure 12.1: A combining tree: threads enter at the leaves and climb towards the counter at the root.

How it works

- Threads **climb the tree from leaves towards the root, combining** with other concurrent operations along the way.
- Every time operations of 2 threads are combined in an internal node, **one thread wins and the other loses**:
  - **Loser** — waits at that node until a return value is delivered to it.
  - **Winner** — proceeds towards the root carrying the **sum of all underlying operations**.
- The winner that reaches the root **adds its sum to the counter**, then **descends the tree distributing a return value to each waiting loser**.

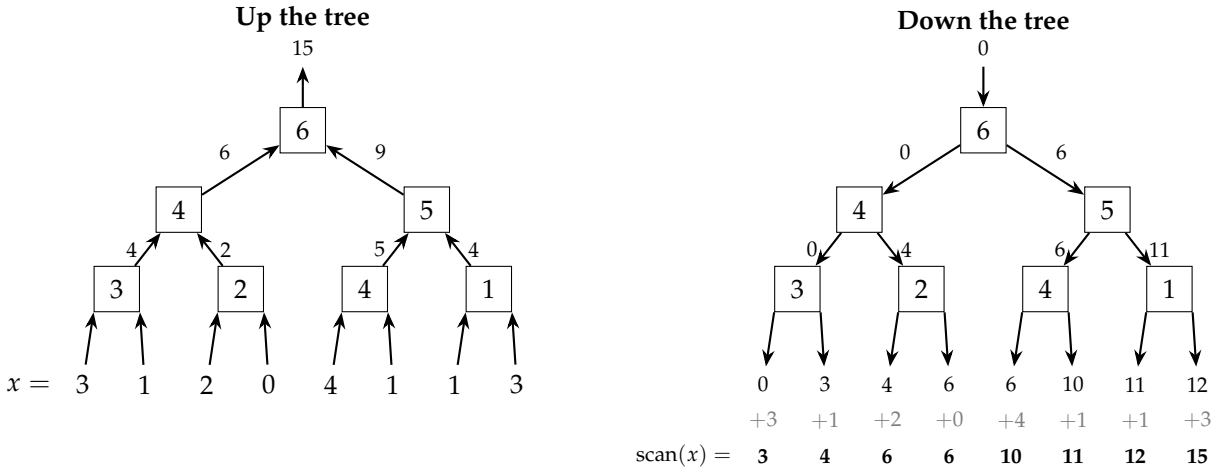


Figure 12.2: Combining as a parallel prefix (scan): sums flow up; offsets flow down so each leaf learns the value the counter had "before" its own increment (after J. Demmel, CS267).

Pros	Cons
More parallelism: increments are parallel until the root is updated	Parallelism **reduces as threads go up the tree**
p leaves take $O(\log p)$ to get a value	p is typically **predefined**; requires $O(p)$ states/memory
Effective speedup $O(p/\log p)$	Threads arriving after p need to wait; **not efficient under low contention** (a lone thread still pays $O(\log p)$)

12.3 Non-blocking Techniques

Definition

In a non-blocking algorithm the **delay of one thread does not cause the delay of others**. By definition, such algorithms **cannot use locks** (a lock holder that is descheduled delays everyone).

12.3.1 Non-blocking Progress Conditions

The three progress conditions

1. Wait-freedom — a wait-free operation is guaranteed to **complete after a finite number of its own steps**, **independent of others**.
2. Lock-freedom — a lock-free operation guarantees that after a finite number of its own steps, **some** operation completes — **not necessarily itself**.
3. Obstruction-freedom — an obstruction-free operation is guaranteed to complete within a finite number of its own steps **after it stops encountering interference** from other operations.

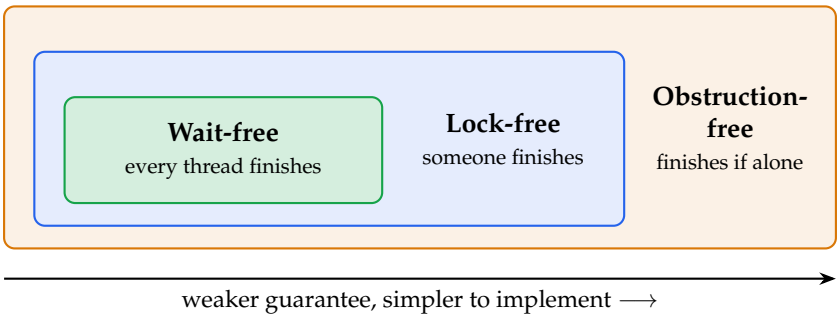


Figure 12.3: Wait-free ⊂ lock-free ⊂ obstruction-free.

Relationships

- **Wait-freedom is stronger than lock-freedom; lock-freedom is stronger than obstruction-freedom.**
- Stronger progress conditions are **harder to implement**; weaker guarantees are generally **simpler**.
- A shared counter **cannot be implemented to be wait-free or lock-free without hardware support**, e.g. **Compare-and-Swap (CaS)**.

12.3.2 Compare-and-Swap (CaS)

```
bool CaS(L, E, N) {
   atomically {
      if (*L == E) {
         *L = N;
         return true;
      } else
         return false;
   }
}
```

Universality

There exists a **wait-free implementation for any concurrent data structure** in a system that supports CaS (Herlihy's universality result). CaS atomically compares location `L` with an expected value `E` and, only if they match, writes the new value `N`.

12.3.3 Shared Counter Based on CaS

```
public class Counter {
  private long value;

  public long getAndIncrement() {
    temp = value;
    while (CaS(value, temp, temp + 1) == false)
        temp = value;            // someone else got in first: re-read and
    retry
    return temp;
  }
}
```

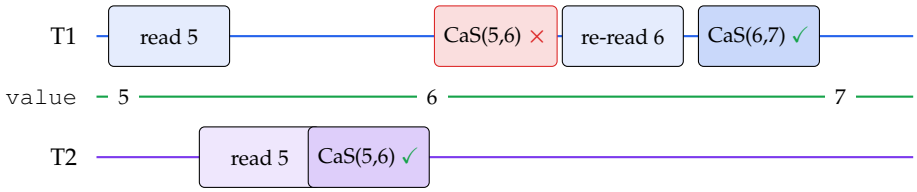


Figure 12.4: A failed CaS means someone else succeeded — so the system as a whole always makes progress.

Properties of the CaS counter

- The solution is **only lock-free** (not wait-free): one unlucky thread can fail forever while others keep succeeding.
- **Backoff** is needed under heavy contention (retry after a short random delay).
- There is still **high contention for a single location**.

12.4 Blocking vs Non-blocking Methods

Problem cases: removing from an **empty** pool; adding to a **full** (bounded) pool.	
Blocking (total)	the caller **waits** until the state changes
Non-blocking (partial)	the method **throws an exception** (or returns a failure code)

12.5 A Bounded Queue (Two Locks)

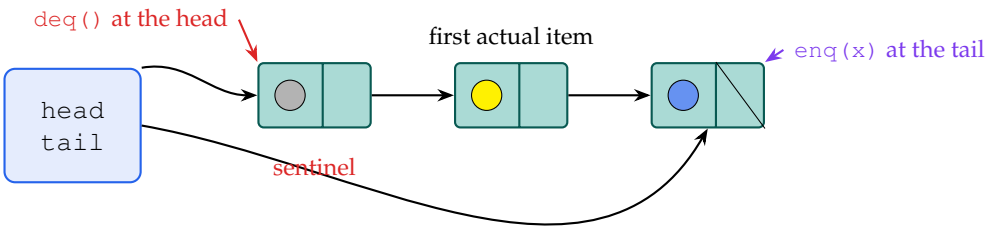


Figure 12.5: A linked queue with a sentinel node. `enq()` and `deq()` work at **different ends** of the object.

Concurrency challenge

Because enqueuers work at the tail and dequeuers at the head, they *could* proceed in parallel. **But what if the queue is empty or full?** Then head and tail are next to each other and the two sides must coordinate.

Design

- The queue always starts with a **sentinel** node; `head` points to it and `tail` points to the last node. The first actual item is `head.next`.
- **`deqLock`** — locks out other `deq()` calls. **`enqLock`** — locks out other `enq()` calls.

Enqueuers and dequeuers **don't share a lock**.

- To tell whether the queue is full or empty: **`permits`**, an atomic counter = permission to enqueue that many more items (e.g. 8). **Decremented by `enq()`, incremented by `deq()`.**

12.5.1 The Enqueuer

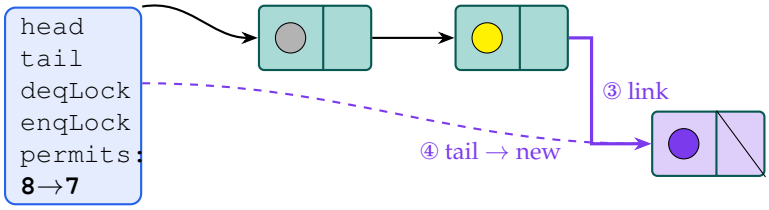


Figure 12.6: Steps of a successful enqueue.

① Lock `enqLock`; read `permits` (OK if > 0).

② No need to lock `tail` — only enqueuers ever touch it.

③ Enqueue the node (link it after the last node).

④ Swing `tail` to the new node.

⑤ `permits.getAndDecrement()`.

⑥ Release `enqLock`.

⑦ If the queue **was empty**, lock `deqLock` and notify/signal waiting dequeuers.

Unsuccessful enqueuer

If the enqueuer reads `permits == 0`, the queue is **full** — "uh-oh". It must wait on `notFullCondition` until a dequeuer frees a slot.

12.5.2 The Dequeuer

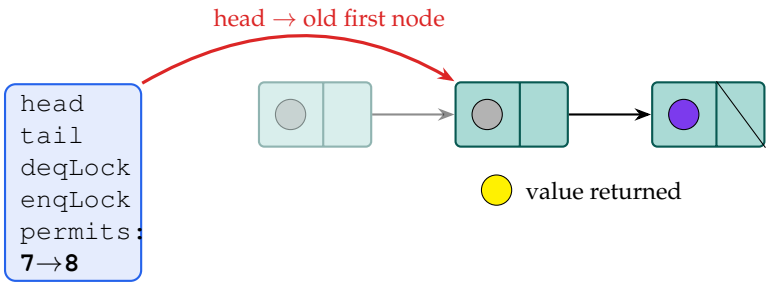


Figure 12.7: Steps of a successful dequeue: the old first node becomes the new sentinel.

① Lock `deqLock`.

② Read the sentinel's `next` field (OK if non-null).

③ Read the value.

④ Make the first node the **new sentinel**.

⑤ Increment `permits`.

⑥ Release `deqLock`.

Unsuccessful dequeuer

If the sentinel's `next` field is `null`, the queue is **empty** — "uh-oh". The dequeuer must wait on `notEmptyCondition`.

12.5.3 The Code Skeleton

```
public class BoundedQueue<T> {
  ReentrantLock enqLock, deqLock;
  Condition notEmptyCondition, notFullCondition;
  AtomicInteger permits;
  Node head;
  Node tail;
  int capacity;
  enqLock = new ReentrantLock();
  notFullCondition = enqLock.newCondition();
  deqLock = new ReentrantLock();
  notEmptyCondition = deqLock.newCondition();
}
```

A sketch of enq (after Herlihy & Shavit)

```
public void enq(T x) {
  boolean mustWakeDequeuers = false;
  enqLock.lock();
  try {
    while (permits.get() == 0)
      notFullCondition.await();          // full: wait (releases enqLock)

    Node e = new Node(x);
    tail.next = e; tail = e;
```

```
    if (permits.getAndDecrement() == capacity)
      mustWakeDequeuers = true;          // queue was empty before this
    enq
  } finally { enqLock.unlock(); }
  if (mustWakeDequeuers) {
    deqLock.lock();
    try { notEmptyCondition.signalAll(); }
    finally { deqLock.unlock(); }
  }
}
```

`deq()` is symmetric: lock `deqLock`, `while (head.next == null) notEmptyCondition.await();`, take `head.next.value`, advance `head`, `permits.getAndIncrement()`, and if the queue was full, lock `enqLock` and `notFullCondition.signalAll()`.

12.6 Monitor Locks and Conditions

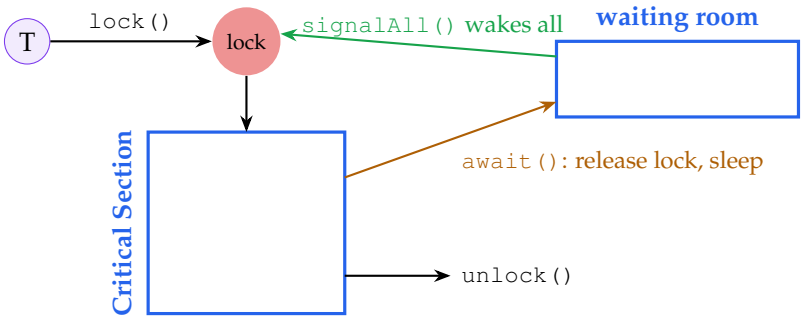


Figure 12.8: A monitor lock with a condition: `await()` releases the lock and moves the thread to the waiting room.

The story in the slides

1. A dequeuer locks, calls `deq()`, finds the queue **empty**, calls `await()` and goes to the waiting room. Another dequeuer does the same.
2. An **enqueuer to the rescue**: it locks, enqueues an item, calls `signalAll()` and unlocks. Both dequeuers wake.
3. **Monitor signalling**: an awakened thread **might still lose the lock to an outside contender**.
4. One dequeuer gets the lock and **finds the item**; the other gets the lock later, finds the queue **still empty** and must wait again — *a dollar short and a day late*. (Hence `while`, not `if`, around `await()` — as in Lecture 9.)

12.6.1 The Lost Wake-Up Problem

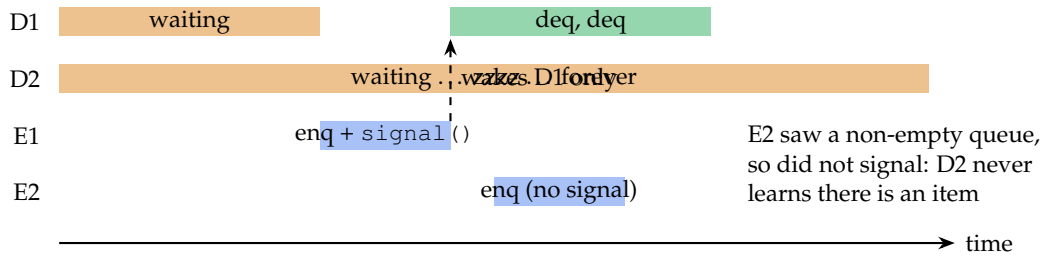


Figure 12.9: Lost wake-up: two items were enqueued but only one waiter was ever woken.

What's wrong here?

Two dequeuers wait. Enqueuer 1 adds an item and calls `signal()` (wakes **one** dequeuer). Before that dequeuer runs, Enqueuer 2 adds another item — the queue is not empty, so it doesn't signal. One dequeuer takes *both*... or takes one and leaves the other item, while the second dequeuer **sleeps forever with an item in the queue**.

Solution to lost wake-up

Always use `signalAll()` and `notifyAll()`, not `signal()` and `notify()`.

12.7 Split Counter

The remaining bottleneck

The `enq()` and `deq()` methods **share no locks** (good) ... but they **do share an atomic counter**, `permits`, accessed on every method call (not so good). Can we alleviate this bottleneck?

Split the counter

- The `enq()` method **decrements only** and **cares only if the value is zero**.
- The `deq()` method **increments only** and **cares only if the value is capacity**.

So keep two counters:

- the enqueuer decrements **`enqSidePermits`**; the dequeuer increments **`deqSidePermits`**;
- when the enqueuer runs out, it **transfers** permits from the deq side;
- **intermittent synchronization** only — but the transfer needs **both locks** (careful about lock ordering!).

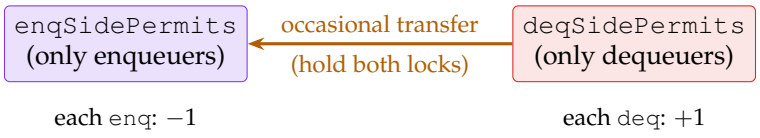


Figure 12.10: Splitting `permits` so the two ends rarely touch the same memory.

12.8 A Lock-Free Queue

Idea (Michael & Scott)

Same sentinel-based linked list, but **no locks**: every update of a shared pointer is done with **CAS** (compare-and-set). An enqueue has two steps — a **logical enqueue** (CAS the last node's `next` to the new node) and a **physical enqueue** (CAS `tail` to the new node).

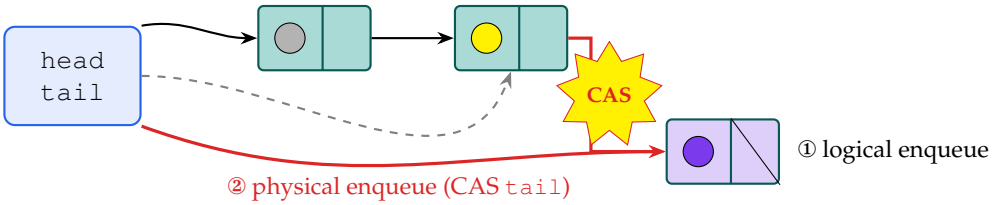


Figure 12.11: Lock-free enqueue: CAS the `next` pointer, then CAS `tail`.

```
public void enq(T value) {
    Node node = new Node(value);
    while (true) {
        Node last = tail.get();
        Node next = last.next.get();
        if (last == tail.get()) {                          // still consistent?
            if (next == null) {                            // tail really is last
                if (last.next.compareAndSet(next, node)) {   // logical
   enqueue
                    tail.compareAndSet(last, node);          // physical
   enqueue
                    return;
                }
            } else {                                       // tail is lagging
   behind
                tail.compareAndSet(last, next);            // help fix it
            }
        }
    }
}
```

These two steps are not atomic

The `tail` field refers to either the **actual last node** (good) or the **penultimate node** (not so good). **Be prepared!** What do you do if you find a **trailing tail**? **Stop and help fix it**: if the tail node has a non-null `next` field, CAS the queue's `tail` field to `tail.next`.

When CASs fail

During the logical enqueue — abandon hope and **restart**. It is **still lock-free**: a failed CAS means *another* thread's enqueue succeeded, so the system made progress.
During the physical enqueue — **ignore it**. A failure means some other thread has already advanced `tail` (helping), which is exactly what we wanted.

12.8.1 Lock-Free Dequeue

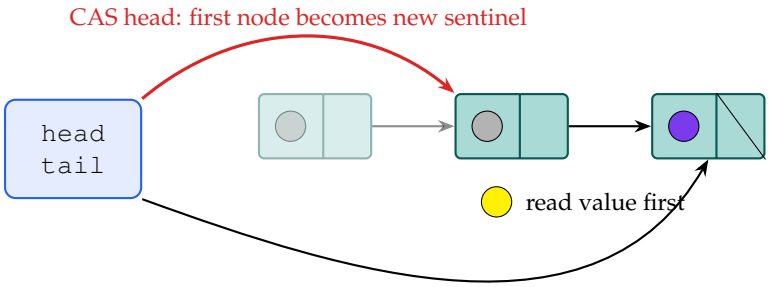


Figure 12.12: Lock-free dequeue: read the value of `head.next`, then CAS `head` forward.

A sketch of the lock-free `deq` (Michael–Scott)

```
public T deq() throws EmptyException {
    while (true) {
        Node first = head.get();
        Node last  = tail.get();
        Node next  = first.next.get();
        if (first == head.get()) {
            if (first == last) {                          // empty, or tail
    lagging
                if (next == null) throw new EmptyException();
                tail.compareAndSet(last, next);           // help the
    enqueuer
            } else {
                T value = next.value;                     // read before
    CAS
                if (head.compareAndSet(first, next))      // first node →
    new sentinel
                    return value;
            }
        }
    }
}
```

12.9 Summary

Lecture Summary

- Locks give safety and liveness but **serialise** code (Amdahl) and cause **memory contention** on the lock variable.
- **Coarse-grained** locking locks the whole structure; **fine-grained** locks parts.
- A **combining tree** lets increments combine on the way to the root: $O(\log p)$ latency, $O(p/\log p)$ speedup, but poor under low contention and needs p fixed in advance.
- **Non-blocking** algorithms use no locks. **Wait-free** (every op finishes in finite own steps) $\supset$ **lock-free** (some op finishes) $\supset$ **obstruction-free** (finishes if alone).

- **CaS** is universal: any object has a wait-free implementation with CaS. The CaS counter is lock-free (retry loop), needs backoff.
- The **bounded two-lock queue** uses a sentinel, separate `enqLock`/`deqLock`, an atomic `permits` counter, and conditions `notFull`/`notEmpty`.
- Awakened threads can lose the lock to outsiders — re-test in a loop. Avoid the **lost wake-up** by using **`signalAll`/`notifyAll`**.
- A **split counter** removes the shared `permits` hot spot.
- The **lock-free queue** enqueues with two CASes (logical then physical); threads that see a lagging `tail` **help** fix it.

Self-check Questions

1. Why do locks hurt performance on a cache-coherent multiprocessor even when there is no logical conflict?
2. Explain how a combining tree computes `getAndIncrement()` for p threads, and give its pros and cons.
3. Define wait-free, lock-free and obstruction-free, and order them by strength.
4. Write `getAndIncrement()` using CaS. Why is it lock-free but not wait-free?
5. In the bounded queue, why can enqueuers and dequeuers usually run in parallel? When can't they?
6. What is the role of the sentinel node?
7. Describe the lost wake-up problem and its fix.
8. What is a split counter and why does it help?
9. In the lock-free queue, why are failed CASes during the logical and physical enqueue handled differently?
10. What does it mean for a thread to "help" another in the lock-free queue?

LECTURE 13

Solution Patterns for Parallel Programming

Overview

How to *design* a parallel algorithm rather than just code one. The lecture introduces Foster's **methodical design** (partition → communicate → agglomerate → map), the main design issues, and a catalogue of reusable **solution patterns — loop parallel, fork/join, divide & conquer, producer/consumer, pipeline, asynchronous agents** — grouped into the three ways to parallelise (**by data, by task, by order**). It closes with **load balancing**: task queues and work stealing. Some slides adapted from Dr. Dilum Bandara and Dr. Srinath Perera.

13.1 Building a Solution by Composition

Reduce to known problems

We often solve problems by **reducing them to a composition of known problems** — finding the way to Habarana (a sequence of known roads), sorting 1 million integers (a known sort, possibly split into known sub-sorts and a merge). Parallel patterns are the "known problems" of parallel programming.

13.2 Designing Parallel Algorithms

No simple recipes

- Parallel algorithm design is **not easily reduced to simple recipes**.
- The **parallel version of a serial algorithm is not necessarily optimal**.
- Good algorithms require **creativity**.

Goal: suggest a **framework** within which parallel algorithm design can be explored, and develop **intuition** as to what constitutes a good parallel algorithm.

13.2.1 Methodical Design (Foster's PCAM)

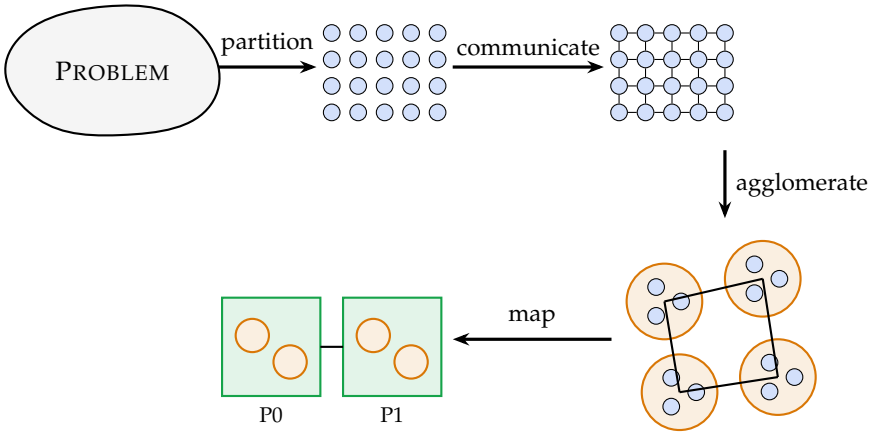


Figure 13.1: Foster's methodology (after Dr. Dobb's "Designing Parallel Algorithms"). Partitioning and communication focus on **concurrency and scalability**; agglomeration and mapping focus on **locality** and other performance issues.

The four steps

1. Partitioning — decompose the computation/data into **small tasks/chunks**. Focus on recognising opportunities for parallel execution; practical issues such as the number of CPUs are **ignored**.
2. Communication — determine the communication required to **coordinate task execution**; define communication structures and algorithms.
3. Agglomeration — evaluate the task and communication structures with respect to **performance requirements** and **implementation costs**. If necessary, **combine tasks into larger tasks** to improve performance and reduce development costs.
4. Mapping — assign each task to a processor, balancing the competing goals of **maximising processor utilisation** and **minimising communication costs**. **Static mapping** at design/compile time; **dynamic mapping** at runtime by load-balancing algorithms.

PCAM applied to the MPI trapezoidal rule (Lecture 10)

P: one task per trapezoid. **C**: every task sends its area to an "add" task. **A**: group n/p consecutive trapezoids per process (and replace the many sends by one reduction). **M**: one block per MPI process (static mapping).

13.2.2 Parallel Algorithm Design Issues

Issue	Notes
Efficiency	useful work per processor per unit time
Scalability	behaviour as problem size and processor count grow
Partitioning computations	**domain decomposition** (based on **data**); **functional decomposition** (based on **computation**)
Locality	**spatial** (nearby data used together) and **temporal** (same data reused soon)
Communication	synchronous vs asynchronous
Agglomeration	combine tasks to **reduce communication**
Load balancing	keep every processor busy

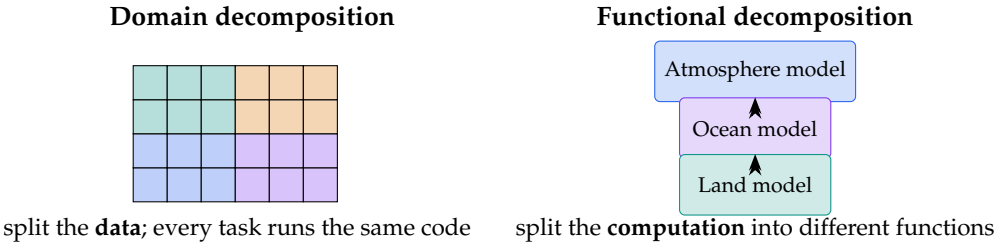


Figure 13.2: Two ways to partition (the climate-model example is the classic illustration of functional decomposition).

13.2.3 Efficiency = Maximising Parallelism?

What we are after

Run the algorithm on the **maximum number of threads** with **minimal communication/waiting**; when the problem grows, the algorithm can handle it by **adding new resources**. This is achieved by the **right architecture + tuning**, and there is no clear recipe. Just like **design patterns** for OOP, people have identified **parallel programming patterns**.

13.3 Solution Patterns for Parallelism

The catalogue

Loop parallel · Fork/join · Divide and conquer · Producer/consumer · Pipeline · Asynchronous agents

13.3.1 Loop Parallel

Definition

If each iteration in a loop **only depends on that iteration's results + read-only data**, each iteration can run in a different thread. As it is based on data, this is also called **data parallelism**.

```
int[] A = .. int[] B = .. int[] C = ..
for (int i; i<N; i++){
    C[i] = F(A[i], B[i]);
}
```

Which of these are loop parallel?

```
// (a)
for (int i; i<N; i++){
    C[i] = F(A[i], B[i-1]);
}
```

```
// (b)
for (int i; i<N; i++){
    C[i] = F(A[i], C[i-1]);
}
```

Answer

(a) Yes. `B[i-1]` is **read-only** input; iteration i writes only `C[i]`. No iteration depends on another's result.
(b) No. Iteration i reads `C[i-1]`, which iteration $i-1$ writes: a **loop-carried dependence**. The iterations must run in order (unless `F` has special structure, e.g. a sum, which can be turned into a parallel prefix/scan).

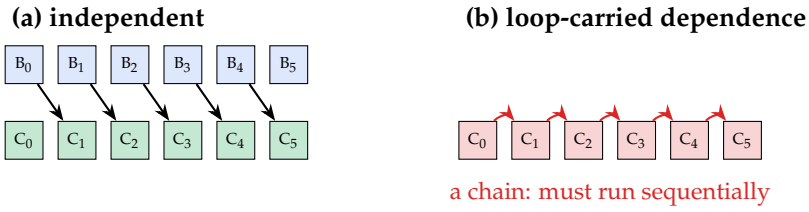


Figure 13.3: Dependence arrows decide whether a loop is parallel.

Implementing Loop Parallel (OpenMP)

```
#  pragma omp parallel for num_threads(thread_count) \
      reduction(+: approx)
   for (i = 1; i <= n-1; i++)
      approx += f(a + i*h);
   approx = h*approx;
```

The `reduction` clause

`approx` is shared by all iterations, which would be a race. `reduction(+: approx)` gives each thread a private copy and adds them together at the end — the "accumulate privately, combine once" idiom from Lecture 4, done for you.

13.3.2 Fork/Join

Definition

Fork a job into smaller tasks (independent if possible), perform them, and **join** them. Examples: calculate the **mean across an array**; a **tree walk**. How to partition? **By data** (e.g. SPMD) or **by task** (e.g. MPSD — multiple programs, single data).

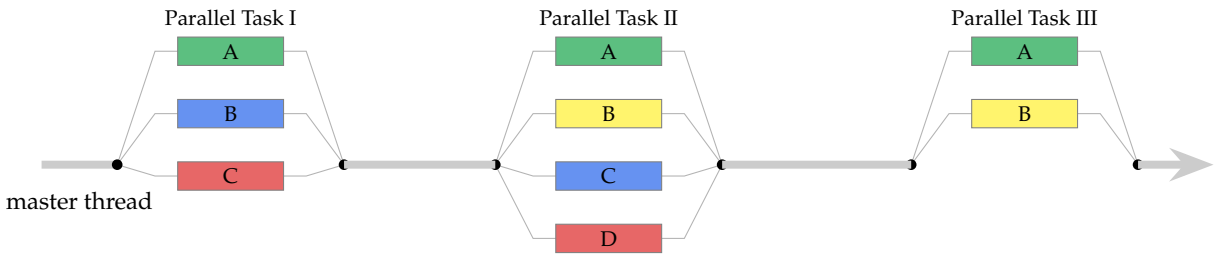


Figure 13.4: The fork–join model: the master thread repeatedly forks a team of tasks and joins them (after Wikipedia, *Fork–join model*).

Fork/join design concerns

Size of work unit: small units ⇒ thrashing (overhead dominates); **big units ⇒ imbalance**. **Balancing load among threads: static allocation** if data/task is completely known (e.g. matrix addition); **dynamic allocation** (e.g. tree walks) using **task queues** or **work stealing**.

Implementing Fork/Join

```c
/* Pthreads */
for (thread = 0; thread < thread_count;
     thread++)
   pthread_create(&thread_handles[
    thread], NULL,
       Hello, (void*) thread);
/* ... later: pthread_join each ... */
```

```c
/* OpenMP */
#  pragma omp parallel \
      num_threads(thread_count)
   Hello();    /* implicit join
    */
```

13.3.3 Divide and Conquer

Definition

Break the problem into **recursive sub-problems** and assign them to different threads. Examples: **quicksort**; **searching for a value in a tree**; **calculating the Fibonacci sequence**. Sub-problems often **fork again**, leading to an **execution tree** (recursion). There may or may not be a join step.

Tree shape

Deep tree ⇒ thrashing (too many tiny tasks). **Shallow tree ⇒ under-utilisation**. In practice: recurse in parallel down to a **cut-off** size, then solve sequentially.

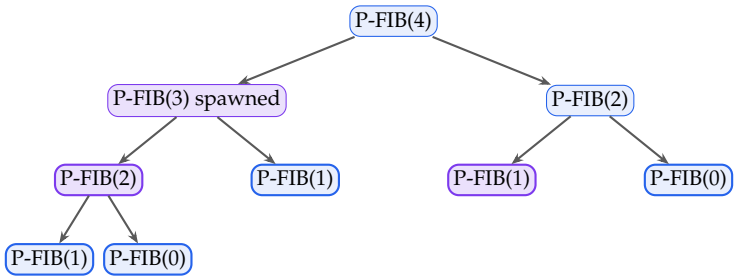


Figure 13.5: Divide and conquer for Fibonacci: each `spawn` (purple) may run on another thread; `sync` is the join (Cormen et al., *Introduction to Algorithms*, 3rd ed.).

```
P-FIB(n)
    if n <= 1 return n
    else
        x ← spawn P-FIB(n - 1)      // fork
        y ← P-FIB(n - 2)
        sync                        // join
        return x + y
```

13.3.4 Producer/Consumer

Definition

Producers put work items in a **shared queue**; consumers take them out. Often used because it helps **dynamically balance the workload**. Example: **crawling the Web** — place newly discovered links in a queue so other threads can pick them up.

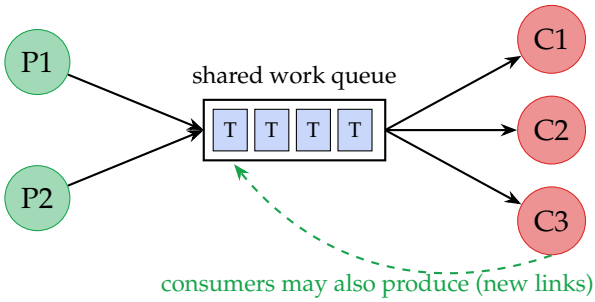


Figure 13.6: Producer/consumer through a shared queue.

13.3.5 Pipeline

Definition

Break a task into **small steps** (which may have dependencies) and assign the execution of the steps to **different threads**. Work is **handed off from step to step**. Example: **read file** → **sort file** → **write to file**.

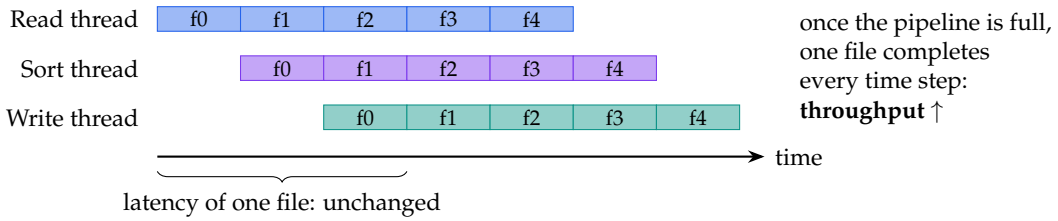


Figure 13.7: A three-stage pipeline processing files f0–f4.

Where the gain comes from

Each individual task **doesn't gain**; but if there are **many instances** of the task, we get **better throughput**. The gain comes from **tuning**: e.g. if read/write are slow but sort is fast, add **more threads to read/write** and fewer to sort.

- **Long pipeline ⇒ high throughput**; **short pipeline ⇒ low latency**.
- Passing data between stages: **message passing** or **shared queues**.

13.3.6 Asynchronous Agents

Definition

The task is done by a set of **agents** working in a **P2P fashion**, with **no clear structure**. They talk to each other via **asynchronous messages**. Example: **detecting storms using weather data** — many agents, each knowing some aspect of storms. Weather events are sent to them, which in turn fire other events, leading to detection.

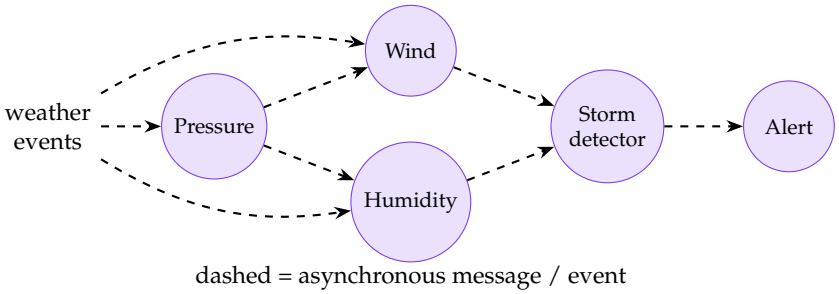


Figure 13.8: Asynchronous agents: no central control; events trigger further events.

13.4 Three Ways to Parallelise

	By data	By task	By order
Idea	partition **data** and give it to different threads	partition the **task into smaller tasks**	partition the task into **steps**
Model / patterns	SPMD; **loop parallel**	**task parallel**, **divide & conquer**	**pipeline**, **asynchronous agents**
Use when	data can be processed locally with few dependencies on other data	work splits into independent or separable sub-tasks	there are temporal (before/after) dependencies
Granularity risk	large units ⇒ under-utilisation; small ⇒ thrashing	too many tasks ⇒ thrashing; too few ⇒ under-utilisation	stage imbalance limits throughput
Example	processing geographical data	quicksort, tree search	read–sort–write

Dependencies among tasks (by task)

- **Removable** — eliminated by **code transformations** (e.g. privatising a temporary, renaming).
- **Separable** — **accumulation** operations (average, sum, count) and **extrema** (max, min): compute partial results per task and combine at the end (a reduction).

By order: dependencies are **temporal** (before/after).

13.5 Load Balancing

The problem

Some threads will be busy while others are idle. Counter it by **distributing load equally**.

- When the cost of the problem is well understood this is possible statically (e.g. matrix multiplication, a known tree walk).
- Other problems are not that simple: it is hard to predict how the workload will be distributed ⇒ use **dynamic load balancing**, which requires **communication between threads/tasks**.

Two methods for dynamic load balancing: **task queues** and **work stealing**.

13.5.1 Task Queues

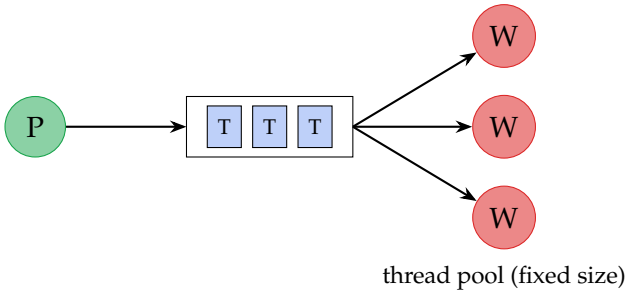


Figure 13.9: A task queue feeding a fixed pool of worker threads.

Task queues

- Multiple instances of task queues (**producer–consumer**).
- Threads come to the task queue **after finishing a task** and grab the next task.
- Typically run with a **thread pool** with a fixed number of threads.

13.5.2 Work Stealing

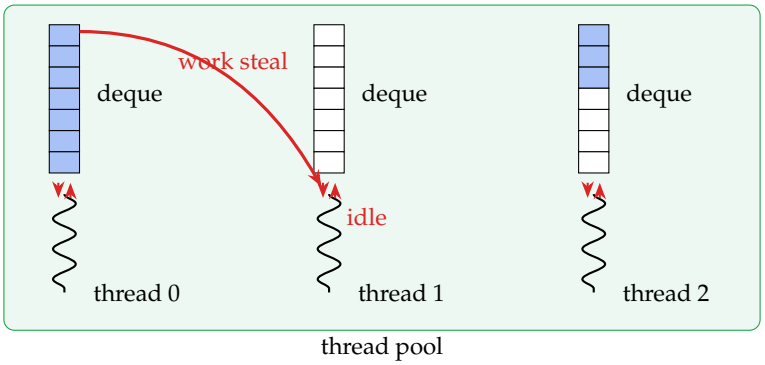


Figure 13.10: Work stealing: an idle thread takes work from the far end of a busy thread's deque (after karlsenchoi.blogspot.com).

Work stealing

- **Every thread has its own work/task queue** (usually a double-ended queue).
- A thread pushes and pops work at **its own end** (cheap, no contention).
- When one thread runs out of work, it goes to **another thread's queue and "steals"** work from the other end.

Used by Java's `ForkJoinPool`, Cilk, Intel TBB.

	Task queue (shared)	Work stealing (per-thread)
Queues	one (or few) shared	one per thread
Contention	every take touches the shared queue	only when stealing
Locality	tasks go anywhere	a thread keeps working on its own (related) tasks
Best for	independent, uniform tasks	recursive fork/join, irregular trees

13.6 Summary

Lecture Summary

- Solve problems by **composing known patterns**; a parallel version of a serial algorithm is not necessarily optimal.
- **Foster's PCAM**: **partition** (find concurrency, ignore CPU count), **communicate**, **agglomerate** (combine tasks to cut overhead), **map** (static or dynamic).
- Design issues: efficiency, scalability, **domain vs functional decomposition**, locality, (a)synchronous communication, agglomeration, load balancing.
- **Loop parallel**: iterations depend only on themselves + read-only data (data parallelism; OpenMP `parallel for`, `reduction`).
- **Fork/join**: split, run, join; watch work-unit size (thrashing vs imbalance).
- **Divide & conquer**: recursive forking; deep tree thrashes, shallow tree under-utilises.
- **Producer/consumer**: shared queue, dynamic balancing.
- **Pipeline**: stages on different threads; throughput not latency; long pipeline = throughput, short = latency.
- **Asynchronous agents**: P2P agents exchanging asynchronous events.
- Parallelise **by data**, **by task**, or **by order**.
- **Dynamic load balancing**: shared **task queues** with a thread pool, or per-thread deques with **work stealing**.

Self-check Questions

1. Name the four steps of Foster's methodology and say which focus on concurrency and which on locality.
2. Distinguish domain decomposition from functional decomposition with an example of each.
3. Which of `C[i] = F(A[i], B[i-1])` and `C[i] = F(A[i], C[i-1])` is loop parallel? Why?
4. What does OpenMP's `reduction` clause do and why is it needed in the trapezoid loop?
5. What happens if fork/join work units are too small? Too big?
6. Why does a pipeline improve throughput but not latency? How would you tune a read–sort–write pipeline?

7. Give an example of a problem suited to asynchronous agents.
8. Classify loop parallel, divide & conquer and pipeline as by data, by task or by order.
9. Compare task queues with work stealing.

LECTURE 14

Embarrassingly Parallel Problems

Overview

Problems that split into independent pieces with almost no communication, and the programming model built to exploit them at scale: **MapReduce**. The lecture covers higher-order functions, the **map** and **reduce** functions, the key–value data model (split, shuffle/sort, merge), **Apache Hadoop**, execution on a cluster, design choices (number of map/reduce tasks, combiners), and worked examples: **word count, mean, inverted index, distributed grep** and sorting. Some slides adapted from Dr. Dilum Bandara and Dr. Srinath Perera.

14.1 Embarrassingly Parallel Problems

Definition

A.k.a. **delightfully parallel** problems. They

- can be **easily parallelised**;
- usually use **simple communication patterns**;
- usually work **without much communication among each other**.

The **MapReduce** programming model provides a powerful abstraction to handle embarrassingly parallel problems.

Typical examples

Rendering independent frames of an animation; Monte Carlo simulations with independent random samples; the primality-testing case study of Lecture 1; brute-force password search; processing millions of log lines or web pages independently. (Speedup is close to linear because the serial fraction in Amdahl's law is tiny.)

14.2 MapReduce

What it is

- A **common pattern** to solve parallel problems, based on 2 constructs from **functional programming**: **map** and **reduce**.
- **Introduced by Google**: Dean & Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004.
- **Extensible** for different applications; **scales to a very large number of nodes**; **hides details like failures** from users.

14.2.1 Higher-Order Functions

Functions as values

Languages such as Java (traditionally) pass **data** as parameters and results of functions. **Higher-order functions** pass **both data and functions** as parameters or results — e.g. Python, Ruby, JavaScript.

```
def f(x):
    return x + 3

def g(function, x):
    return function(x) * function(x)

print(g(f, 7))       # f(7) = 10, so prints 100
```

```
# the two built-ins MapReduce is named after
from functools import reduce
squares = list(map(lambda x: x*x, [1, 2, 3, 4]))     # [1, 4, 9, 16]
total   = reduce(lambda a, b: a + b, squares)        # 30
```

14.2.2 The Two Functions

MapReduce accepts 2 functions as inputs

1. Map function $Y \leftarrow fn_1(X)$ — accepts an input X and outputs another Y (in practice, a list of **(key, value)** pairs).
2. Reduce function $Z \leftarrow fn_2(\text{List}\langle Y \rangle)$ — accepts an array of Ys and returns another output Z.

What the implementation does

Given a list of inputs, the MapReduce implementation:

1. applies the **map** function to each entry in the list, which **emits (key, value) pairs**;
2. **collects the results, groups them by key**, and passes each group to the **reduce** function as an array.

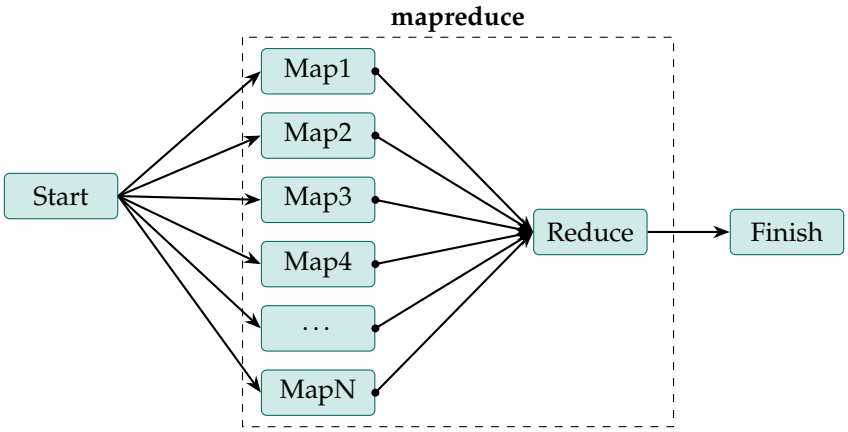


Figure 14.1: MapReduce as a computation graph: many independent maps feed one (logical) reduce.

14.2.3 Example: Word Counting

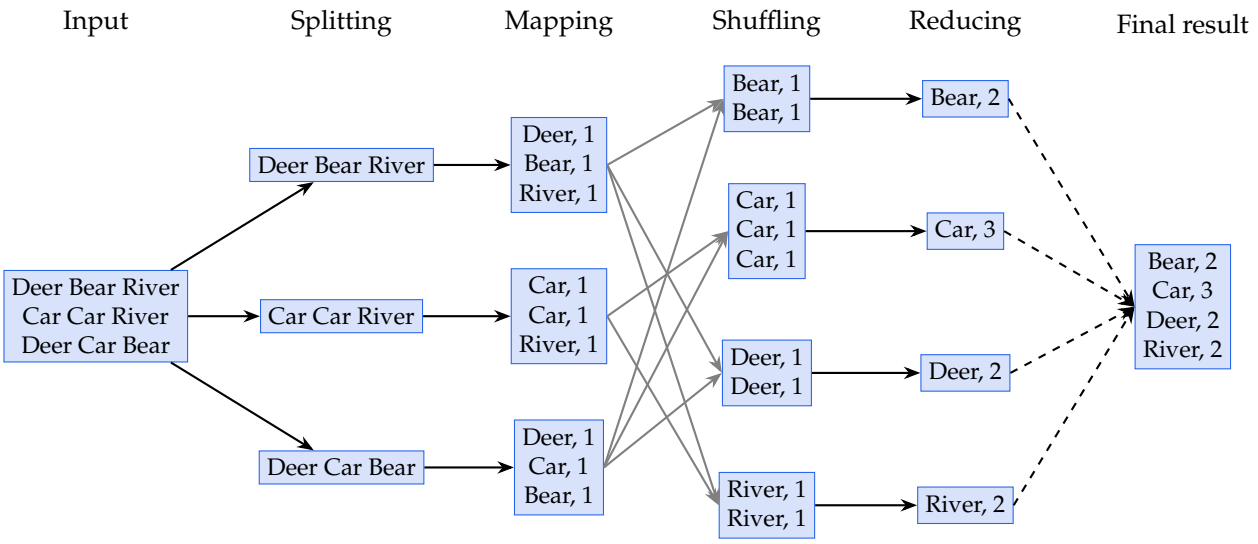


Figure 14.2: MapReduce word count: split, map to (word, 1), shuffle by key, reduce by summing (after xiaochongzhang.me).

How to do this for a large dataset using a distributed system?

That is exactly the job of a MapReduce *framework*.

14.2.4 Why MapReduce?

The value of the framework

- Implementing the same pattern in a distributed system **isn't easy**: you need to worry about **communication, failures, initialisation**, etc.
- **MapReduce frameworks worry about all of those.** You write the **map and reduce functions** and call the framework.
- It **forces you to think parallel at design time**; gives a **higher level of abstraction**; is **very generic** and covers many use-cases (see the Hadoop "PoweredBy" list).

14.2.5 MapReduce Implementations

Ways to implement

- **In-memory** implementation (threads on one machine).
- **Distributed** implementation, with communication **by messages**, **by the file system**, or **by databases**.

Communication requirements: only **broadcast and reduce** operations are needed — which is why it fits embarrassingly parallel problems so well (compare `MPI_Bcast` and `MPI_Reduce` in Lecture 10).

14.3 MapReduce with Hadoop

Apache Hadoop

Apache Hadoop is an open-source implementation of MapReduce. It **handles all details about distributed execution**; you just give it the **map and reduce functions**. (Its storage layer, HDFS, splits large files into blocks across the cluster.)

14.3.1 The Data Model

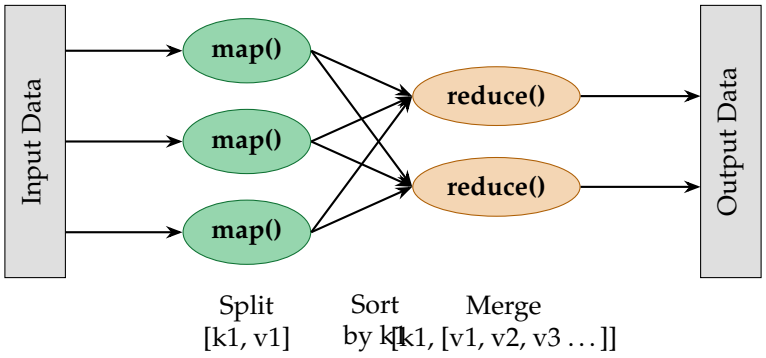


Figure 14.3: The MapReduce data model: maps emit (key, value) pairs; the framework sorts by key and hands each key's values to a reducer.

What Hadoop does, step by step

1. Breaks the input data into multiple data items **by new lines**, and runs the **map function once for each data item**.
2. When executed, the map function **outputs (key, value) pairs**.
3. Hadoop **collects all (key, value) pairs**, **sorts them by key**, and **groups values with the same key together**.
4. For each **distinct key**, Hadoop runs the **reduce function once**, passing the key and the list of values for that key.
5. The reduce function outputs (key, value) pairs, and Hadoop **writes them to a file** as the final result.

14.3.2 Execution on a Cluster/Cloud

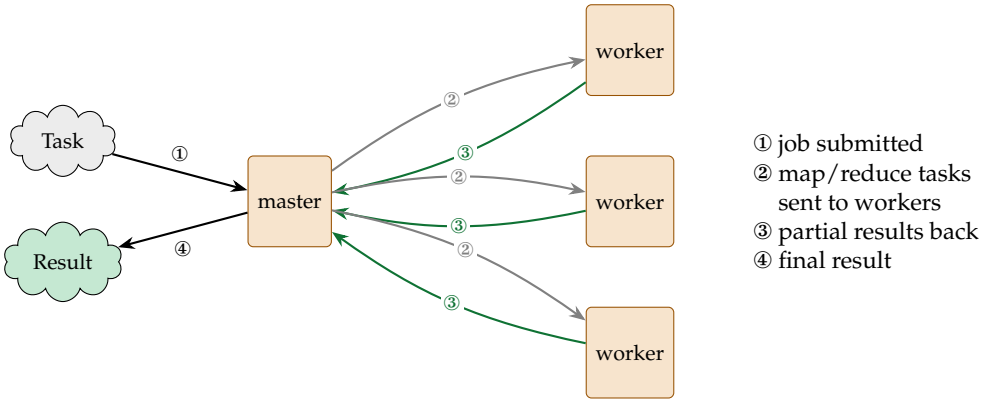


Figure 14.4: Execution on a cluster: the master farms tasks out to workers and collects the results (after cbsolution.net).

14.3.3 MapReduce Execution (Dean et al.)

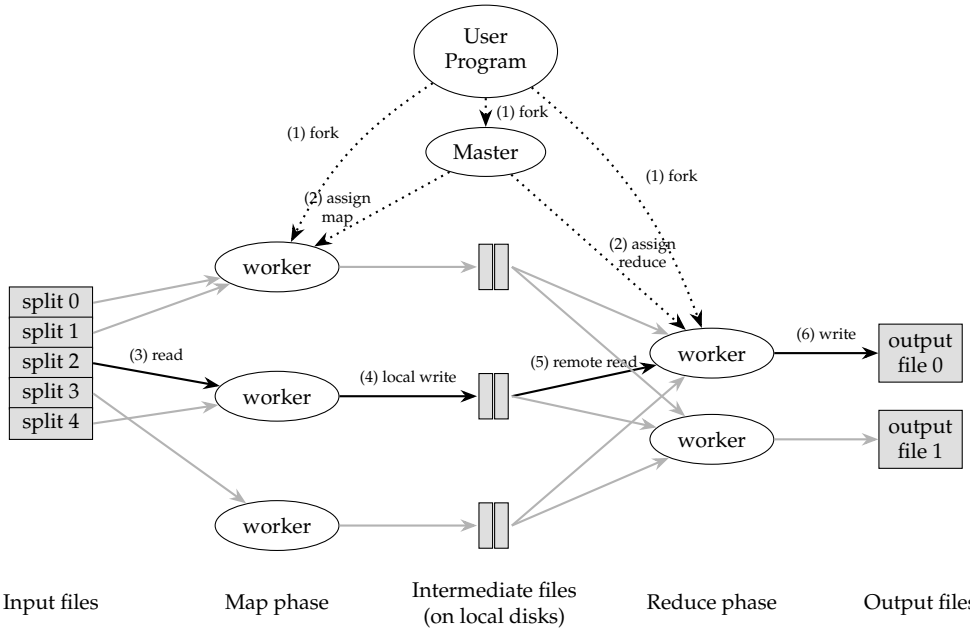


Figure 14.5: MapReduce execution overview (redrawn from Dean & Ghemawat, OSDI 2004).

The six steps

1. The user program **forks** a master and many workers.
2. The master **assigns map tasks** and **reduce tasks** to idle workers.
3. A map worker **reads** its input split and runs the map function.
4. Intermediate (key, value) pairs are **written to the local disk**, partitioned into R regions (one per reducer, e.g. by $hash(key) \bmod R$).
5. Reduce workers **remotely read** their region from every map worker, sort by key, and run reduce.
6. Each reducer **writes** its own output file.

If a worker fails, the master simply **re-executes its tasks** elsewhere — possible because map and reduce are side-effect-free functions.

14.4 Designing MapReduce Applications

Granularity and combiners

- You control **task granularity** by changing the number of map and reduce tasks: how many map tasks? how many reduce tasks?
- **Fine grain** ⇒ **more parallelism** ⇒ **more communication overhead**, and vice versa.
- Usually frameworks handle **load balancing and failures**.
- If there are a large number of maps, you need a **combine function** as well.

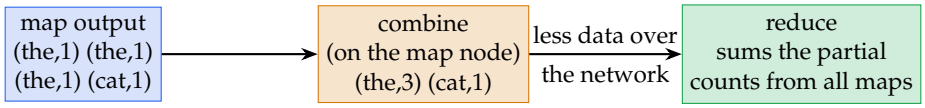


Figure 14.6: A combiner is a "mini-reduce" run locally on each mapper's output.

14.5 Examples

Problems to think about

- **Sorting** — how to sort an array of 1 million integers using MapReduce?
- **Inverted index** — a normal index maps document → terms; an inverted index maps **terms** → **documents**. With a million documents, how do we build it with MapReduce?
- **Frequency distribution of word occurrences** — count occurrences and build a histogram.

14.5.1 Word Count

Find the words in a collection of documents and their frequency of occurrence.

```
Map(docId, text):
    for all terms t in text
        emit(t, 1);

Reduce(t, values[]):
    int sum = 0;
    for all values v
        sum += v;
    emit(t, sum);
```

14.5.2 Mean

Compute the mean value associated with the same key.

```
Map(k, value):
    emit(k, value);

Reduce(k, values[]):
    int sum = 0;
    int count = 0;
    for all values v
        sum += v;
        count += 1;
    emit(k, sum/count);
```

Careful with combiners here

Mean is **not associative**: the mean of means is not the overall mean. To use a combiner, have the map (or combiner) emit **(sum, count)** pairs and let the reducer add sums and counts before dividing.

14.5.3 Inverted Index

```
Map(docid, text):
    for all word w in text
        emit(w, docid);

Reduce(w, docids[]):
    emit(w, docids[]);
```

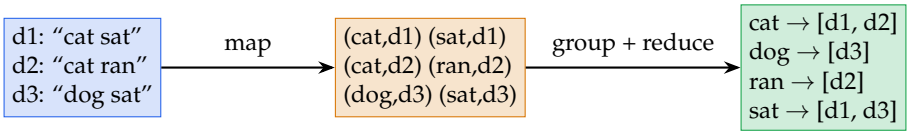


Figure 14.7: Building an inverted index (term → documents).

14.5.4 Distributed Grep

```
map(k, v):
    Id docId = ..  (read file name)
    if (v matches grep pattern)
        emit(k, (pattern, docid));

Reduce(k, values[]):
    emit(k, values[]);          // identity reduce
```

Sorting with MapReduce (one approach)

Map emits **(number, null)**; the framework's shuffle **sorts by key**; with a **range partitioner** (reducer j gets keys in the j-th range) and an identity reduce, concatenating the reducers' output files gives the sorted array. This is how the TeraSort benchmark works on Hadoop.

14.6 Summary

Lecture Summary

- **Embarrassingly (delightfully) parallel** problems split into independent pieces with simple, infrequent communication.
- **MapReduce** (Google, OSDI 2004) is built on the functional constructs **map** and **reduce**; it scales to huge clusters and hides failures.
- **Higher-order functions** take functions as arguments — the programmer supplies map and reduce to the framework.
- Map: $X \to$ list of (key, value); the framework **groups by key** (shuffle/sort); Reduce: (key, list of values) $\to$ output.
- Implementations: in-memory or distributed (messages, file system, databases); only **broadcast and reduce** communication is needed.
- **Hadoop**: splits input by lines, runs map per item, sorts and groups pairs by key, runs reduce once per distinct key, writes results to files.
- Design: choose the number of map/reduce tasks (granularity vs overhead); use a **combiner** when there are many maps.
- Examples: **word count, mean, inverted index, distributed grep**, sorting.

Self-check Questions

1. What makes a problem "embarrassingly parallel"? Give two examples.
2. What is a higher-order function? Evaluate `g(f, 7)` for the example in the notes.
3. Describe the roles of the map function, the shuffle/sort, and the reduce function.
4. Trace word count on the input "Deer Bear River / Car Car River / Deer Car Bear".
5. What does a MapReduce framework handle that you would otherwise have to write yourself?
6. Why is only broadcast and reduce communication needed?
7. What is a combiner, and why can't you use the mean reducer directly as a combiner?
8. Write map and reduce functions for an inverted index and for distributed grep.
9. How could you sort 1 million integers using MapReduce?

APPENDIX A

Quick Reference

A.1 MPI Cheat Sheet

Call	Purpose
`MPI_Init(&argc, &argv)` / `MPI_Finalize()`	start / stop MPI
`MPI_Comm_size(comm, &p)` / `MPI_Comm_rank(comm, &r)`	number of processes / my rank
`MPI_Send(buf, n, type, dest, tag, comm)`	point-to-point send (may buffer or block)
`MPI_Recv(buf, n, type, src, tag, comm, &st)`	point-to-point receive (always blocks)
`MPI_Get_count(&st, type, &n)`	how many elements arrived
`MPI_Reduce(in, out, n, type, op, root, comm)`	combine to root
`MPI_Allreduce(in, out, n, type, op, comm)`	combine, result to all
`MPI_Bcast(buf, n, type, root, comm)`	root's data to all
`MPI_Scatter(sb, sn, st, rb, rn, rt, root, comm)`	split root's array into blocks
`MPI_Gather(sb, sn, st, rb, rn, rt, root, comm)`	collect blocks on root
`MPI_Allgather(sb, sn, st, rb, rn, rt, comm)`	collect blocks on all

```
mpicc -g -Wall -o prog prog.c        # compile
mpiexec -n 4 ./prog                  # run with 4 processes
```

A.2 Deadlock at a Glance

Strategy	Technique	Note
Ignore	ostrich algorithm	UNIX, Windows
Detect & recover	cycle detection / E, A, C, R matrices; preemption, rollback, kill	needs periodic checking
Avoid	safe states, Banker's algorithm	needs max claims in advance
Prevent	spool (ME), request all (H&W), take away (NP), **order resources** (CW)	ordering is the practical one

A.3 Progress Conditions

Condition	Guarantee	Example
Wait-free	every operation completes in a finite number of its own steps	(universal construction with CAS)
Lock-free	some operation completes in a finite number of steps	CAS counter, Michael–Scott queue
Obstruction-free	an operation completes if it runs alone long enough	simple optimistic retry
Blocking (lock-based)	none if a lock holder is delayed	two-lock bounded queue

A.4 Pattern Selector

If the problem looks like…	Use	Parallelise by
independent loop iterations	loop parallel (OpenMP `parallel for`)	data
split, work, combine	fork/join	data or task
recursive sub-problems	divide & conquer	task
a stream of work items of unknown cost	producer/consumer + task queue	task
a sequence of stages over many items	pipeline	order
loosely-coupled event processing	asynchronous agents	order
huge, independent records	MapReduce	data

A.5 Building These Notes

```
pdflatex main.tex      # run twice so the contents and figure list fill in
pdflatex main.tex
# or simply:  latexmk -pdf main.tex
```

Uses the same preamble and packages as Part 1 (TikZ, pgfplots, tcolorbox, listings, mathpazo, helvet, courier, pifont); all diagrams are drawn in TikZ, so no image files are needed. Figures adapted from slides credit their original sources in the caption.